Cache and Memory Performance Profiling on an Apple M5 Max

ECSE 4320/6320 Class-wide Project 2

[Member A], [Member B], [Member C]

October 2026

The benchmark code is in the Project-2 folder of the submitted repository. Appendix A lists the source files. Nothing needs to be run to check the report: every figure, table and number below is embedded in this document.

Summary

This project measures latency, bandwidth, concurrency, cache behavior and address-translation cost on an Apple M5 Max MacBook Pro (macOS, ARM64, 18 cores, 36 GB unified memory) with C benchmarks written for the purpose. Intel MLC, perf, a CPU governor, thread pinning, SMT control and user-selectable huge pages do not exist on this platform. Each was replaced by the closest honest substitute, listed in Section 1.3. Every causal statement is labeled *(measured)*, *(inferred)* from measurements, or *(speculation)*. Statements without a cause are plain measurements.

Table 1. Headline results. Cycle values use an estimated 4.58 GHz effective clock (dependent-add calibration, Section 1.4). Percentages use the 460 GB/s peak that Apple specifies for the M5 Max with 32-core GPU, which is this machine's configuration (Section 1.2).

Quantity	Value	Derived	Note
L1 hit latency (32 KiB)	0.66 ns	3.0 cycles (est.)	random single-cycle chase, 12 rounds
L2 hit latency (2 MiB)	7.12 ns	33 cycles (est.)	128 pages, small translation cost
L2 latency (4 MiB)	7.62 ns	35 cycles (est.)	256 pages, includes about 1.3 ns translation cost
SLC region (64 MiB)	92 ns	421 cycles (est.)	a ramp, not a plateau; SLC size not exposed
DRAM latency (1 GiB)	108.7 ns	498 cycles (est.)	loaded-harness idle probe: 113 ns
One-core sequential read bandwidth	94 GB/s (DRAM), 211 GB/s (L1)	21% of 460 GB/s	NEON, independent accumulators
One-core random-miss concurrency	21.9 GB/s	N = 18.9 lines	Little's Law with one-core idle latency
Chip read bandwidth, maximum	293 GB/s	64% of 460 GB/s	6 threads; flat from 5 to 16 threads
Chip read-modify-write bandwidth	374 GB/s	81% of 460 GB/s	16 threads, application bytes
Knee, two definitions	5 threads (K1), 4 threads (K2)	see Section 5	pre-defined, first crossing and power

Three most important findings.

1. **One core is limited by outstanding misses, the chip by something below the DRAM peak.** One core sustains about 19 outstanding random-line misses (21.9 GB/s at 110 ns), consistent with Little's Law *(measured)*. Streaming reads from all cores saturate at 293 GB/s (64% of peak) with 5 or 6 threads, and more threads only add latency *(measured)*. A read-modify-write mix reaches 374 GB/s (81%) and read plus two non-temporal writer threads moves more total traffic than reads alone, so the read-only plateau is not the DRAM limit. Which internal limit sets it is not established (Section 11).
2. **Benchmark design confounds change the answer by large factors.** The data written by a store kernel matters: plain stores of zero lines run at 277 GB/s against 68 GB/s for random data, and stnp runs 1.6x faster with uniform lines than with random lines. A write-allocate factor of two therefore cannot be read from these data (Section 4). The in-page placement of the lines changes the TLB result: the apparent penalty at 65 536 pages is 16 ns with the original placement and 7 ns once cache-set conflicts are removed (Section 8).

3. **Prefetching decides what 'latency' and 'bandwidth' mean.** A sequential chase at 128 B steps measures 7.8 ns per node against 108 ns for a random chase, 14x lower *(measured)*, and the benefit ends between 1 KiB and 2 KiB strides. The scattered kernel costs 88x the unit-stride cost per element at a 1.5 GiB footprint but only 5.4x at 12 KiB (Sections 3 and 7).

Hardware counters. No cache or TLB miss counters were obtained. The PMU event database for this chip lists the needed events and an own counter-reading tool was written, but reading counters requires administrator rights, which the assistant did not have. Cache and TLB behavior is therefore inferred from timing signatures throughout, and the attempts are logged in Appendix H. This is the main weakness of the project against the assignment's counter requirement.

1. Method and environment

1.1 Approach

All measurements come from one C program (membench) with sub-commands for pointer chasing, streaming kernels, a loaded-latency harness and the TLB and kernel tests. A Python driver runs each experiment in rounds. Within a round every configuration runs once in shuffled order with a fresh random seed, so slow drift cannot align with a sweep axis and each round builds new random permutations. Results are medians over rounds with the minimum and maximum shown. With fewer than 10 rounds an interquartile range is not meaningful, so figures show min to max bands and, for the headline curves, every round as a dot. The number of rounds (n) is listed in every table.

Hypotheses were written before the benchmarks existed and are reproduced in Appendix C. Every later amendment is listed in Appendix D. No benchmark was tuned to match a hypothesis; where a result looked wrong, the benchmark was inspected and, if it was at fault, corrected and the correction recorded (Section 13).

Latency uses a single random cycle through all lines of a footprint (a random permutation closed into one cycle), so every load depends on the previous one and no prefetcher can guess the next address. Streaming bandwidth uses NEON loads and stores with several independent accumulators so that the kernel is limited by memory, not by a dependency chain. All results are consumed (summed into a value that is returned and stored in a global sink) so that the optimizer cannot delete the work (Section 9 shows what happens otherwise).

1.2 Environment

Table 2. Environment and run protocol. Machine metadata (hardware, OS, compiler and flags, QoS, rounds, warm-up, thermal state, frequency) is also saved in machine-readable form next to every raw data file.

Item	Value
Machine	MacBook Pro Mac17,6, Apple M5 Max, 36 GB unified memory
GPU cores and peak memory bandwidth	32-core GPU (system_profiler). Apple's technical specifications list 460 GB/s for the M5 Max with 18-core CPU and 32-core GPU, and 614 GB/s for the 40-core-GPU part. This machine's 36 GB configuration is offered only with the 32-core GPU, so 460 GB/s is the reference used for every percentage.
Cores	18 = 6 Super (perflevel0) + 12 Performance (perflevel1); no efficiency cores, no SMT
L1D	128 KiB per Super core, 64 KiB per Performance core (hw.perflevel*.l1dcachesize)
L2	16 MiB shared by the 6 Super cores, 8 MiB per cluster of 6 Performance cores (hw.perflevel*.l2cachesize, cpusperl2)
System-level cache	size not exposed by sysctl; inferred only from timing
Page size, line size	16 KiB (hw.pagesize), 128 B (hw.cachelinesize)

Item	Value
OS, compiler	macOS 26.5.2 (build 25F84, Darwin 25.5.0); Apple clang 21.0.0 (clang-2100.1.1.101)
Compiler flags	-O2 -std=gnu17 -mcpu=native -ffp-contract=off -Wall -Wextra -Wno-unused-function -pthread
Threads and QoS	each worker sets QOS_CLASS_USER_INTERACTIVE on itself; the main thread sets it before any work. No pinning exists on macOS. Placement is checked from cache signatures (Section 1.5)
Frequency	automatic (no governor); estimated 4.58 GHz effective on a Super core by a dependent-add calibration after a 300 ms warm-up spin at process start (Section 1.4)
Power and thermal	main collection on battery power (about 42% charge, Low Power Mode off, powermode 0), no thermal or performance warning recorded at the start of any round or at the end. An earlier collection under AC power gave the same latencies within 1% (L1 0.66 ns in both; DRAM 109.0 and 108.8 ns), so battery operation did not change the result.
Background load	recorded per group (ps, load average); background daemons (indexing, policy checks) used a few percent of one core at times. Shuffled rounds and medians limit their effect; the maximum values in the tables include such rounds
Rounds per experiment	Exp 1: 12; Exp 2 stride and pattern: 5 and 9; stride aliasing tests: 9 and 7; Exp 3: 9 (single-thread) and 6 (loaded); Exp 4: 12; read-ceiling tests: 7; plateau tests: 9; Exp 5 bandwidth: 9; Exp 6: 5 and 7; Exp 7: 12; Exp 8: 5
Timer	mach_absolute_time, 24 MHz (41.7 ns tick); every timed region lasts at least 25 ms

1.3 Deviations from the assignment

Intel MLC, perf counters, thread pinning, a governor, SMT control and huge pages do not exist or are not accessible on this platform; the SLC size is not exposed and cycle values are estimates. The substitutes and their consequences are listed in Appendix B, Table 10. In short: own benchmarks replace MLC; timing inference replaces counters (No counter values; all cache and TLB statements are inferred from timing (labeled as such)); QoS classes replace pinning; page locality replaces the huge-page comparison; the SLC appears as a latency ramp.

1.4 Frequency and warm-up

A fresh process starts at a low clock and ramps to its steady value over about 100 ms (Section 9, Table 18). Every process therefore spins for 300 ms before any measurement, and each timed region is at least 25 ms. The effective frequency is estimated from a chain of 32 dependent 64-bit adds per iteration, assuming one cycle per add: 4.58 GHz at the start and end of the collection (4.575 and 4.578 GHz). The assumption is not verified. powermetrics was not run: it requires administrator rights and the password is not available to the assistant. A script that runs it during a single-thread spin was prepared. **Frequency cross-check: not done** (open item). All cycle values remain estimates based on the dependent-add calibration.

1.5 Validation

The self-test passes (32 checks): chase cycles visit every node once; the NEON, strided, random-pattern, store and arithmetic kernels match scalar references; the clock and the frequency estimate are within bounds. The checks on the data pass with no flags: latency is non-decreasing with footprint (5% tolerance); L1 latency is 3.0 cycles (inside 2.5 to 6.5); repeatability (interquartile range over median) is 0.4% at 32 KiB, 0.4% at 4 MiB and 1.6% at 1 GiB; no DRAM-resident bandwidth exceeds 460 GB/s (higher values occur only for cache-resident buffers, Section 9); the chase sweep (108.7 ns) and the loaded-harness idle probe (112.7 ns) agree within 4%; one-core N_eff never exceeds the number of chains. A thread with the user-interactive QoS class shows the Super-core signature (96 KiB resident in L1, 12 MiB in L2: 0.66 ns and 10.9 ns), a background-QoS thread a Performance-core signature (96 KiB: 16.4 ns). For more than 6 threads the placement of the extra threads cannot be observed.

2. Experiment 1: zero-queue baselines

Hypotheses (Appendix C): H1a to H1e, HV1, HV2. Expected: L1 3 to 5 cycles, L2 12 to 40 cycles, a latency ramp (not a plateau) for the system-level cache, DRAM 90 to 130 ns, latency non-decreasing with footprint.

What was measured. Time per dependent load for one thread chasing a single random cycle over footprints from 4 KiB to 2 GiB (an otherwise idle machine, one thread, no other memory traffic), after a warm pass, 12 rounds, each round with a new random permutation.

Table 3. Zero-queue latency (random single-cycle chase, one thread). Cycles use the estimated 4.58 GHz clock. The number of 16 KiB pages is listed because address translation contributes to larger footprints.

Level	Working set	16 KiB pages	median ns	min to max ns	cycles (est.)	n rounds
L1	32 KiB	2	0.66	0.65 to 0.72	3	12
L1, 96 KiB (inside Super L1D)	96 KiB	6	0.66	0.65 to 0.66	3	12
L2, 1.5 MiB	1.41406 MiB	91	7.11	7.09 to 7.16	33	12
L2, 2 MiB	2 MiB	128	7.12	7.10 to 7.58	33	12
L2, 4 MiB	4 MiB	256	7.62	7.58 to 8.05	35	12
L2, 12 MiB	12 MiB	768	11.13	8.94 to 13.27	51	12
SLC region, 64 MiB	64 MiB	4096	91.89	83.82 to 100.87	421	12
DRAM, 1 GiB	1 GiB	65 536	108.75	108.42 to 122.01	498	12

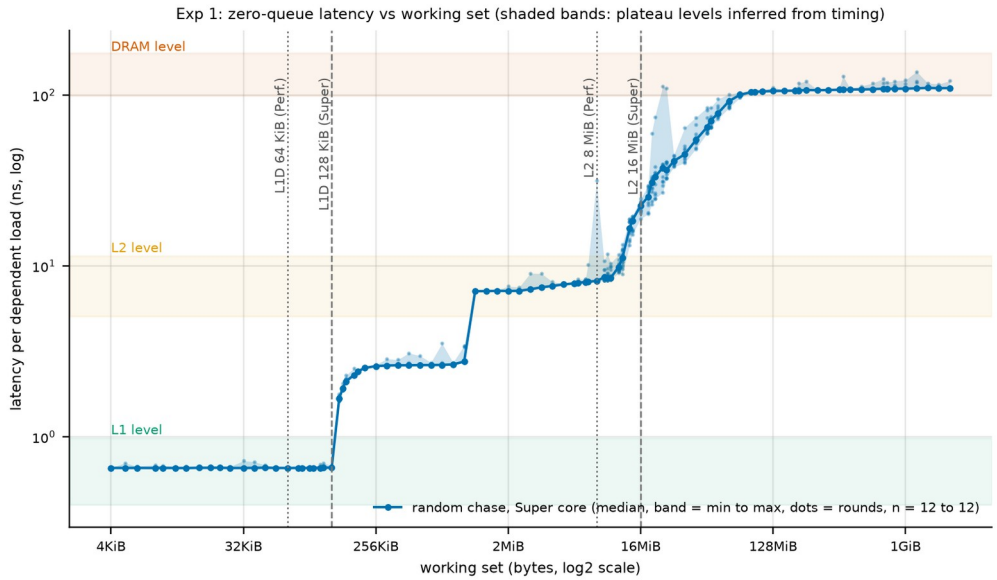


Figure 1. Zero-queue latency versus footprint (log-log). Line: median of 12 rounds; band: minimum to maximum; dots: every round. Shaded bands mark the plateau levels; dotted and dashed vertical lines mark the sysctl capacities of the Super and Performance cores. Knobs fixed: one thread, QoS user-interactive, 128 B nodes, random single cycle. Knob varied: footprint.

Results. L1 latency is 0.66 ns, 3.0 cycles (estimated) *(measured)*. The L2 latency is 7.1 ns at 2 MiB and 7.6 ns at 4 MiB. The 4 MiB point spans 256 pages and likely includes a small translation cost (Exp 7 shows +1.3 ns for 256 pages with one line per page), so the 2 MiB value, 33 cycles (estimated), is the better L2 figure *(inferred)*. At 12 MiB the latency is 11.1 ns, in the ramp toward the system-level cache. DRAM latency at 1 GiB is 108.7 ns (498 cycles, estimated); the separate loaded-latency harness measures 112.7 ns with no bandwidth threads.

Level edges (from the curve, no counters). The first footprint above 1.5 times the L1 plateau is 144 KiB (sysctl capacity 128 KiB). The ramp toward the system-level cache starts at 13.5 MiB, below the 16 MiB L2 capacity, and the curve reaches within 10% of the DRAM value at 76 MiB. Between those points the latency rises smoothly; an SLC capacity cannot be read from it and none is claimed. A single thread cannot use the whole 16 MiB L2 at full speed; why is not established.

Plateau between L1 and L2. Between about 144 KiB and 1.1 MiB the latency is flat at about 2.6 ns, well below the L2 value. It is not explained by a cache listed in `sysctl`. Section 6 tests two explanations.

Caveat. Cycle values depend on the frequency estimate. The 1 GiB point includes translation cost, because the footprint spans 65 536 pages.

3. Experiment 2: access pattern and granularity

Hypotheses: H2a to H2f. Expected: a sequential chase is at least 5 times faster than a random one; a random chase does not depend on node stride; the prefetcher stops helping at large strides; useful bandwidth of a 16 B access at a stride of 128 B or more is about 1/8 of the line rate; and the penalty at 128 B and 256 B strides does not depend on power-of-two alignment.

Setup. (a) A pointer chase whose nodes are `stride` bytes apart, in sequential order or in random order, for strides 64 B to 4 KiB. The number of distinct lines is held at 1 Mi (the footprint is the larger of 1 GiB and 1 Mi times the stride) so that a larger stride never shrinks the cache footprint. (b) Independent loads (not a chain) of 16, 32, 64 or 128 B at the same strides, sequential or random slots, one thread, with eight independent address streams. (c) Aliasing tests: independent random 16 B loads at strides 128, 192, 256, 320, 384, 512, 640 and 1024 B, with and without a hashed line offset inside each stride block, and the two arithmetic kernels of Section 7 at the same strides.

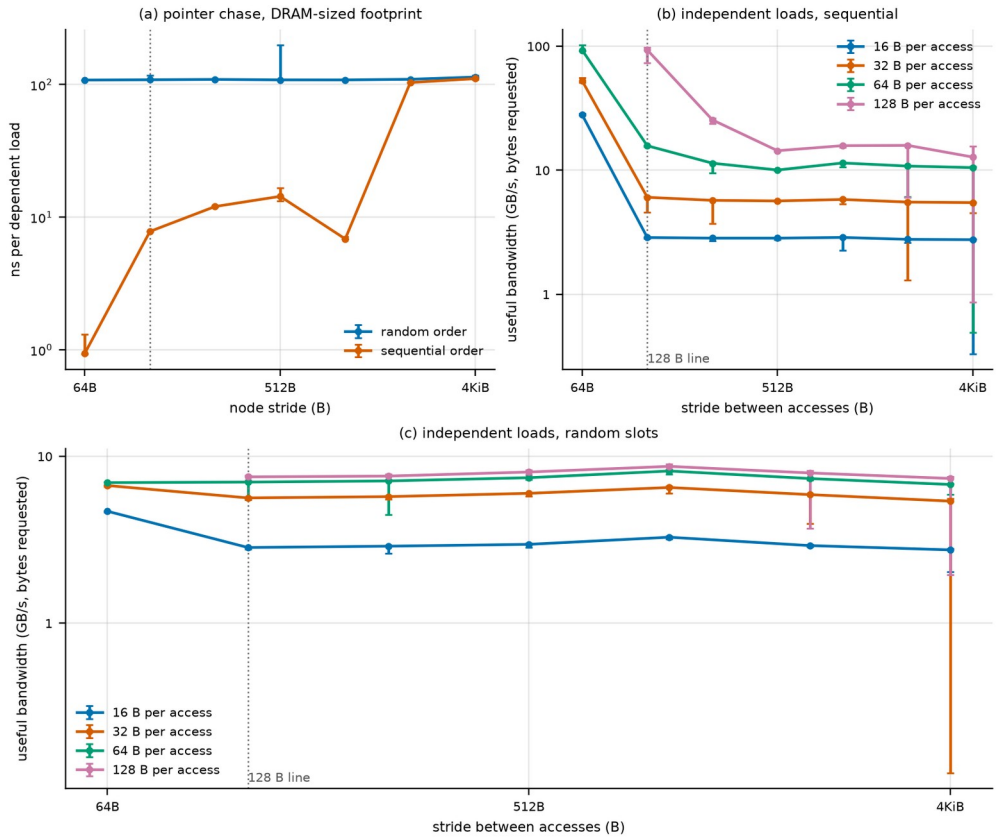


Figure 2. Pattern and granularity. (a) Dependent chase versus node stride, sequential and random order. (b, c) Useful bandwidth of independent loads of four access sizes versus stride, sequential and random slots. Median, bars: minimum to maximum over rounds (n = 5 for the chase, n = 9 for the independent loads). Fixed: one thread, footprint of at least 1 GiB. Varied: stride, order, access size.

Results. A sequential chase with 128 B nodes measures 7.8 ns per node against 108.4 ns for the random chase, 13.9x lower *(measured)*. The random chase does not depend on node stride: 108 to 114 ns over 64 B to 4 KiB (the 4 KiB value is 5% higher, plausibly the translation cost at 4 GiB) *(measured)*. The prefetch benefit of the sequential chase persists up to a 1 KiB stride (6.8 ns) and is gone at 2 KiB (103 ns) and 4 KiB (111 ns), so the hardware tracks strides of at most 1 KiB in this test *(measured)*. The value at 1 KiB (6.8 ns) is lower than at 256 B and 512 B (12.0 and 14.3 ns); this non-monotonic behavior is unexplained (Section 11). Independent sequential loads of a whole line reach 93 GB/s at stride 128 B, close to the one-core read bandwidth of Section 6; the same loads at a 256 B stride reach 25 GB/s and at 1 KiB 16 GB/s. A 16 B access at a 128 B stride yields 2.87 GB/s sequentially and 2.84 GB/s randomly, so the useful rate is about 1/33 and 1/2.6 of the same stream with whole-line accesses, not the 1/8 predicted (H2e).

Interpretation. *(inferred)* Because sequential access is fast only while the stride is within the prefetcher's reach, a "latency" taken from a sequential or small-stride chase measures streaming, not memory latency (Exp 8). *(inferred)* The granularity of memory traffic is the 128 B line. Random 16 B loads complete 177 million times per second, each moving a full line, which is 22.7 GB/s of line traffic, equal to the one-core random-miss plateau of Section 5 (21.9 GB/s). Useful bandwidth of narrow random accesses is therefore the line rate times 16/128. Random whole-line (128 B) loads from the same kernel reach only 7.5 GB/s of line traffic, about a third of that; the cause is unexplained (Section 11).

3.1 Stride aliasing test

The aliasing concern is that power-of-two strides map to few cache sets or memory channels and inflate the cost. Two things were varied: the stride itself (power of two or not) and a hashed offset of the access inside the stride block (which changes the address bits above the offset but keeps the 128 B alignment).

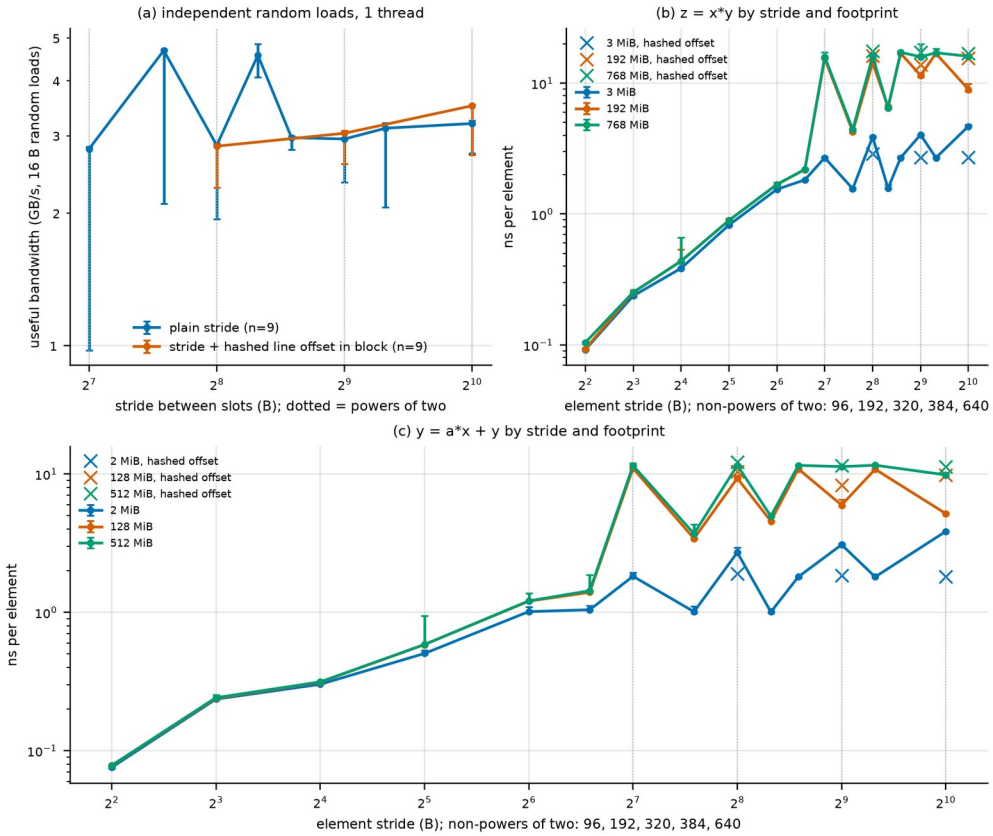


Figure 3. Stride aliasing. (a) Independent random 16 B loads: useful bandwidth versus stride, plain and with a hashed line offset inside each stride block (median, bars: min to max, n = 9). (b, c) Arithmetic kernels z = x*y* and *y* = *a*x + y: ns per element versus element stride for three footprints; x markers: hashed offset (n = 7). Dotted vertical lines mark 128, 256, 512 and 1024 B.

Result. The slow strides are the multiples of 128 B: 128, 256, 384, 512, 640 and 1024 B give 2.8 to 3.2 GB/s, while 192 B (4.7 GB/s) and 320 B (4.6 GB/s) are about 1.7 times faster, although neither is a power of two. 384 B and 640 B are not powers of two and are as slow as 128 B. The penalty therefore follows "stride is a multiple of the 128 B line" (every access lands on the first 16 B of a line), not power-of-two alignment *(measured)*. The hashed line offset changes the power-of-two strides by −0% (256 B), 3% (512 B) and 10% (1024 B), so channel or set aliasing between lines is not the cause of the 128 B and 256 B slowdown *(measured; aliasing between lines is not supported as the cause)*. The kernels show the same pattern: at the 1.5 GiB footprint the z = x*y kernel costs 15.9 ns per element at a 128 B stride and 17.2 ns at 384 B, but only 4.4 ns at 192 B and 6.5 ns at 320 B. Why half-line-offset strides run faster is not established. A plausible reading is that accesses at an offset of 64 B inside a line are served differently from accesses at offset 0 (speculation, untested).

4. Experiment 3: read/write mix

Hypotheses: H3a to H3e. Expected: read-only is fastest; non-temporal stores change little; dc zva is faster than plain stores; a write-allocate model corrects 100% write toward the read rate; store throughput depends on the data stored.

Setup. One thread streams through 10-line groups, reading R lines from one buffer and writing W lines to another: 100% read, 70/30, 50/50, 100% write, with plain stores, with non-temporal pair stores (stnp), with dc zva, and a read-modify-write mix on the same lines. Footprints 256 KiB, 8 MiB and 1 GiB (9 rounds). The store data is random (a per-line pseudo-random sequence) unless stated; variants with a constant value per line ("uniform") and with zero data test the content dependence. A loaded variant runs the same mixes on 0 to 16 threads with the probe thread measuring latency. dc zva zeroes a block without reading it; the block size read from DCZID_EL0 is 64 B, so two instructions clear one 128 B line. "Application GB/s" counts the bytes the program reads and writes. "Modeled DRAM GB/s" multiplies application bytes of plain stores by a write-allocate factor and is a model, not a measurement.

Table 4. Single-thread mixes at 1 GiB. Measured and modeled quantities are in separate columns; the modeled column is not evidence.

mix (1 thread, 1 GiB, random store data)	measured application GB/s	min to max	MODELED traffic factor / modeled DRAM GB/s (not measured)	n
100% read	87.4	81.8 to 91.7		9
70/30, plain stores	101.0	87.6 to 104.0	1.30 / 131	9
50/50, plain stores	108.0	99.2 to 109.4	1.50 / 162	9
100% write, plain stores	68.2	58.8 to 70.4	2.00 / 136	9
70/30, stnp	105.4	101.7 to 110.4		9
50/50, stnp	122.2	105.4 to 124.2		9
100% write, stnp	90.6	81.6 to 91.7		9
50/50 read-modify-write	137.4	128.5 to 139.4		9

Table 5. Content dependence of write throughput at 1 GiB (application GB/s, median and min to max over 9 rounds).

100% write, 1 thread, 1 GiB (application GB/s)	random data	uniform non-zero line	zero data
plain stores	68 (59 to 70)	68 (46 to 74)	277 (177 to 281)
stnp	91 (82 to 92)	142 (118 to 144)	140 (110 to 144)
dc zva (no data stored)			269 (257 to 276)

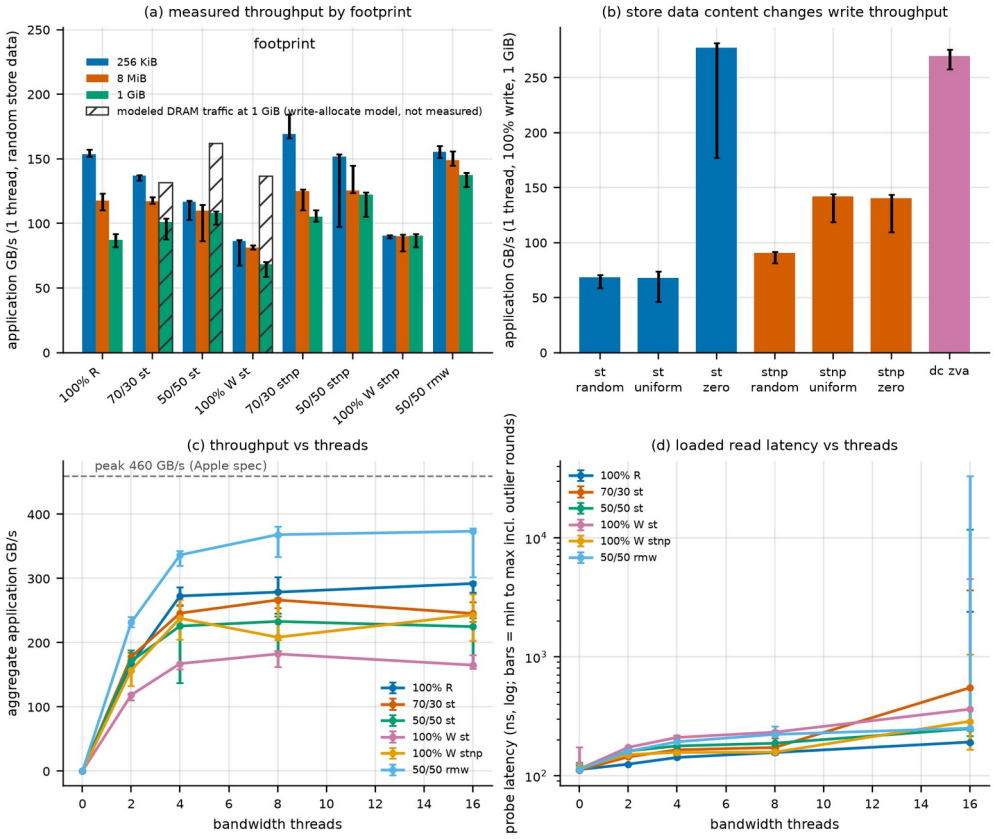


Figure 4. Read/write mix. (a) Single-thread application GB/s by mix and footprint; hatched open bars are the write-allocate MODEL for plain stores at 1 GiB, not measured. (b) Store content test at 1 GiB. (c, d) Loaded runs with 0 to 16 bandwidth threads: aggregate GB/s and probe latency (log axis; bars include rare outlier rounds). Median and min to max over rounds.

Results. Mixed traffic is faster than read-only for one thread: 70/30 reaches 101 GB/s and 50/50 108 GB/s against 87 GB/s for reads alone, and 100% write with plain stores reaches 68 GB/s (0.78 of read). The ordering assumed in H3a (read fastest) does not hold. Read-modify-write reaches 137 GB/s. Non-temporal stores of random data are 1.33x faster than plain stores for 100% write.

The data content matters. Plain stores of zero lines run at 277 GB/s, 4.1 times the rate for random data, and `stnp` of uniform non-zero lines runs at 142 GB/s against 91 GB/s for random lines. `dc zva`, which stores no data at all, reaches 269 GB/s, the same as zero-data plain stores. Single-thread read bandwidth does not depend on buffer content (loaded read runs with random buffers: 284 and 285 GB/s at 4 and 6 threads, against 274 and 293 GB/s). *(inferred)* Zero lines and uniform lines are handled more cheaply somewhere in the write path (for example detected and not written in full); a rate of 269 GB/s from one core is therefore not DRAM write bandwidth, and neither the `dc zva` result nor the uniform `stnp` result can be used as evidence for or against write-allocate traffic. The mechanism is untested.

Write-allocate. With random data, plain 100% write moves 68 GB/s of application bytes; a model in which each stored line is first fetched would put the DRAM traffic at 136 GB/s, more than the one-core read rate of 87 GB/s. The data neither confirm nor exclude write-allocate: the direct test (zero-fill without fetch) is confounded by content, and `stnp` with random data is only 33% faster than plain stores, not the factor of about 2 that removing a fetch would give. A

write-allocate factor of two cannot be read from these data. *(speculation)* Plain stores may be limited by the store buffer or by write-back bandwidth rather than by an allocate fetch.

Why mixed traffic beats read-only for one core. One core's read rate is limited by the number of demand misses it can keep in flight (Exp 4: about 19 random-line misses, while sequential streams reach about 80 lines in flight through the prefetcher). Stores retire into a store buffer and drain through a different path, so adding stores raises the total bytes moved without consuming more read slots. This reading is consistent with the data (read-modify-write, which needs both paths, is fastest at 137 GB/s); it is not proven.

Loaded runs. On 16 threads, 100% read reaches 292 GB/s with a probe latency of 192 ns, read-modify-write reaches 374 GB/s at 251 ns, 70/30 plain stores reach 246 GB/s with a probe latency of 549 ns (4.9x idle), and 100% write with plain stores 165 GB/s. Write-heavy mixes give lower aggregate throughput and higher read latency than read-only, but because their rates differ the comparison of H3d (equal application rate) was not performed.

Caveat. The Table 4 rows with large min-to-max ranges (50/50 plain stores) include rounds disturbed by background activity; medians are used.

5. Experiment 4: intensity sweep, knee and Little's Law

Hypotheses: H4a to H4f, H11. Expected: one core sustains a plateau of outstanding misses; chip read bandwidth saturates at 55 to 85% of 460 GB/s; loaded latency stays within 1.3 times idle up to about half the saturation bandwidth and then rises; the number of lines in flight, N = bandwidth times latency divided by 128 B, is 200 to 700 at the knee.

Setup. (a) One thread runs K independent random chains (K = 1 to 64): throughput should grow linearly with K until a limit of outstanding misses, and N_eff = throughput times the K = 1 latency divided by 128 B tests Little's Law where the true count K is known. (b) A loaded-latency harness runs N sequential-read threads (no injected delay, 1 to 8 threads in steps of one, then 10, 12, 14 and 16) while a separate probe thread measures latency by a random chase over 1 GiB. (c) Injection delay: 16 threads and 6 threads execute a busy loop of the stated length between 1 MiB slices, which lowers the offered load. Each bandwidth thread streams a private 256 MiB buffer. 12 rounds.

Knee definitions, fixed before the analysis. K1: the smallest thread count whose median bandwidth reaches 95% of the maximum median bandwidth of the thread sweep. K2: the point of the whole curve (thread sweep and both delay series) with the largest bandwidth divided by latency. Both are reported because a knee defined by one rule alone can be an artifact of that rule. The 1.5 times idle criterion used elsewhere is not used as a knee.

Table 6. Thread sweep (no injected delay, median of 12 rounds, min to max). Idle latency 113 ns. N = bandwidth times latency / 128 B (Little's Law).

threads	GB/s	min to max	% of max	latency ns	min to max	vs idle	N lines	N per thread	n
1	88	82 to 91	30%	121	119 to 129	1.07x	83	83	12
2	163	151 to 171	56%	126	125 to 133	1.12x	160	80	12
3	229	205 to 242	78%	132	130 to 153	1.17x	235	78	12
4	274	236 to 288	93%	142	141 to 160	1.26x	303	76	12
5	287	190 to 298	98%	149	146 to 296	1.32x	333	67	12

threads	GB/s	min to max	% of max	latency ns	min to max	vs idle	N lines	N per thread	n
6	293	211 to 302	100%	153	151 to 192	1.35x	349	58	12
7	287	230 to 301	98%	156	151 to 197	1.39x	351	50	12
8	266	212 to 303	91%	158	150 to 802	1.40x	328	41	12
10	282	271 to 292	96%	173	160 to 266	1.54x	381	38	12
12	285	267 to 296	97%	201	168 to 303	1.78x	446	37	12
14	287	248 to 297	98%	230	163 to 1657	2.04x	516	37	12
16	289	275 to 297	99%	201	189 to 396	1.78x	453	28	12

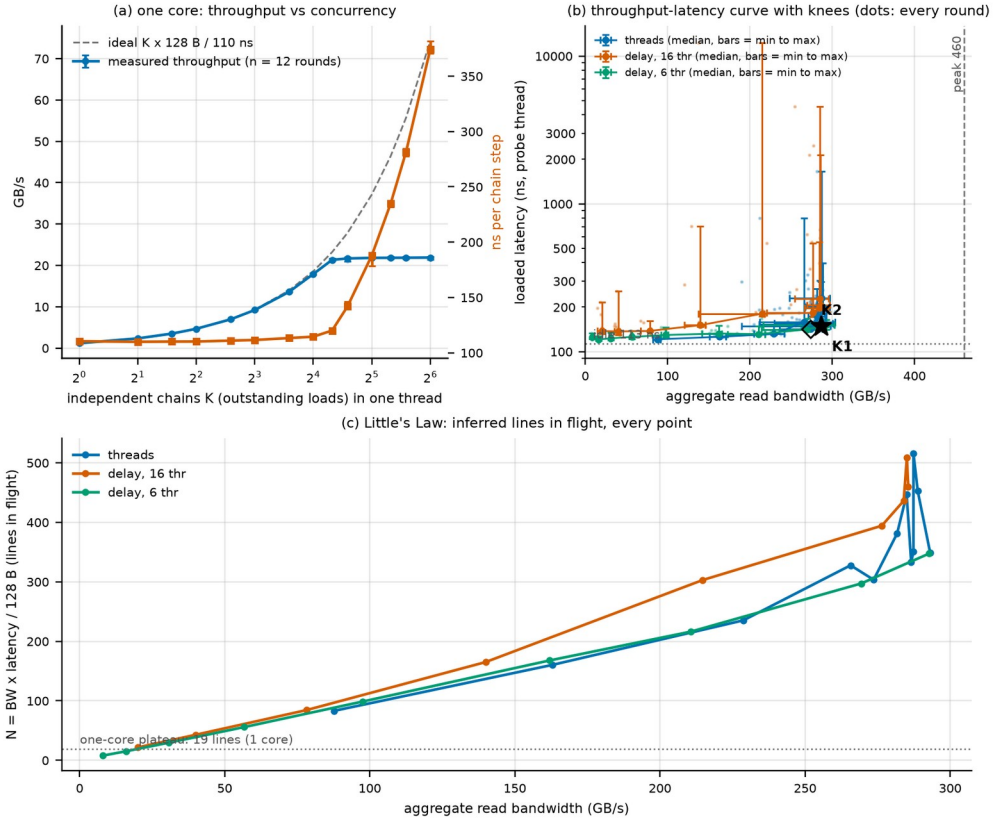


Figure 5. Intensity sweep. (a) One core: throughput (left axis) and latency per chain step (right axis) versus number of independent chains, dashed: ideal K x 128 B / L0. (b) Throughput-latency curve from the thread sweep and both delay series with the two knees marked; points: every round; bars: min to max; log latency axis. (c) Lines in flight N = BW x latency / 128 B for every point. Fixed: sequential reads, 256 MiB per thread, probe over 1 GiB. Varied: threads, injected delay.

One core. Throughput grows linearly with K (K = 1: 1.2 GB/s, 128 B per 110 ns) and stops at 21.9 GB/s from about K = 20; beyond that the latency per step rises in proportion to K. N_eff equals K for every K up to the plateau (largest N_eff/K ratio 1.00) and then stays at 18.9 lines *(measured)*. This validates the Little's Law estimate in the one case where the in-flight count is known. One core can keep about 19 random-line demand misses outstanding *(inferred)*.

Chip. Bandwidth rises from 88 GB/s (1 thread) to 274 GB/s (4 threads, 93% of the maximum median of 293 GB/s) and is flat, between 287 and 293 GB/s, from 5 to 16 threads. The maximum is 64% of 460 GB/s, inside the 55 to 85% range of H4b, but it is reached with 5 or 6 threads rather than 8 to 14. **K1 is 5 threads** (287 GB/s, 149 ns, 1.32x idle) and **K2 is 4 threads** (274 GB/s, 142 ns, 1.26x idle). The two definitions differ by one thread and 13 GB/s; both place the knee at 4 to 5 threads, with latency 1.26 to 1.32 times idle *(measured)*. Latency is 1.40x idle at 8

threads and 1.78x at 12, and exceeds 2 times idle only at 14 threads (2.04x), where bandwidth no longer increases: the threads beyond the knee add latency only, as predicted for a saturated resource. H4c as phrased (latency at least doubling at saturation) is refuted: at the point of maximum bandwidth the latency is 1.35x idle.

Little's Law at every point. N rises from 83 lines (1 thread) to 349 lines (6 threads) while bandwidth rises from 88 to 293 GB/s, and keeps rising to about 453 lines at 16 threads while bandwidth is flat; the extra requests queue. The in-flight count per thread falls from 83 (1 thread) to 28 (16 threads). A single sequential stream holds about 83 lines in flight, far more than the 19 lines of the random demand-miss test, so the sequential streams are supplied by the hardware prefetcher *(inferred)*. The assumption in H4f that N per thread stays near the one-core random plateau is therefore refuted. In the delay series N per thread falls with the injected delay as expected (Table 16).

Read-only ceiling tests. Read-only streaming flattens at 293 GB/s while the read-modify-write mix reaches 374 GB/s, so the read plateau is not the DRAM limit. Four cheap tests were run.

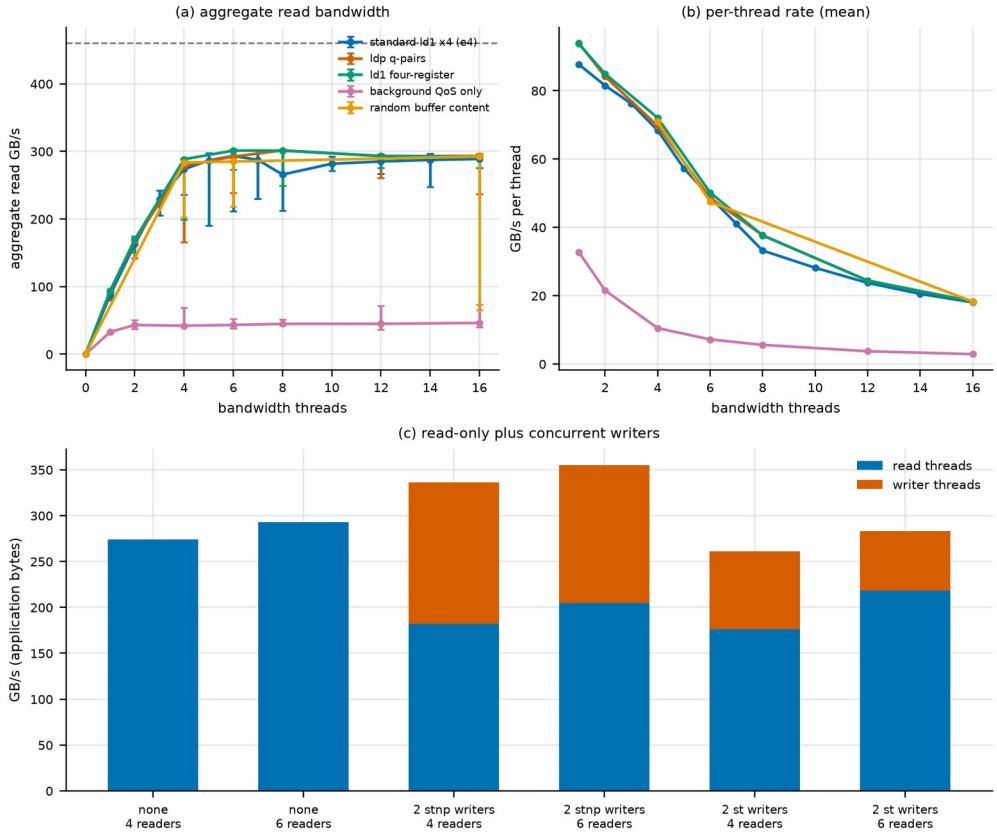


Figure 6. Read-only ceiling tests. (a) Aggregate read bandwidth versus threads for the standard kernel, ldp q-pair loads, ld1 four-register loads, background-QoS threads and random buffer content. (b) Mean per-thread rate. (c) Two concurrent writer threads (stnp or plain stores, random data) added to 4 or 6 reader threads: reader bandwidth plus writer bandwidth (n = 7 for writers; 12 for the baseline). Median and min to max.

The load instruction mix does not matter: ldp and ld1 four-register kernels reach 292 to 301 GB/s, the same as the standard kernel *(measured)*. Random buffer content does not matter *(measured)*. Threads of background QoS reach only 33 GB/s for one thread and 46 GB/s for 16, far below

interactive threads (a different core class or a throttle; not separated) *(measured)*. Two concurrent `stnp` writer threads raise the total traffic to 336 GB/s (4 readers) and 355 GB/s (6 readers), above the read-only plateau, while the readers lose bandwidth; two plain-store writers do not raise it (261 and 282 GB/s, below the read-only 274 and 293 GB/s) *(measured)*. The tests exclude the load instruction mix, the buffer content and the DRAM peak as the cause of the 293 GB/s plateau. They do not identify it. Candidate causes such as a limit on read-return bandwidth or on outstanding requests are untested speculation.

6. Experiment 5: working-set sweep

Hypotheses: H5a and the placement checks. Expected: L1 edge at 128 KiB, L2 edge at 16 MiB, DRAM plateau before 512 MiB; one-core read bandwidth falls from the L1 level to about 100 GB/s.

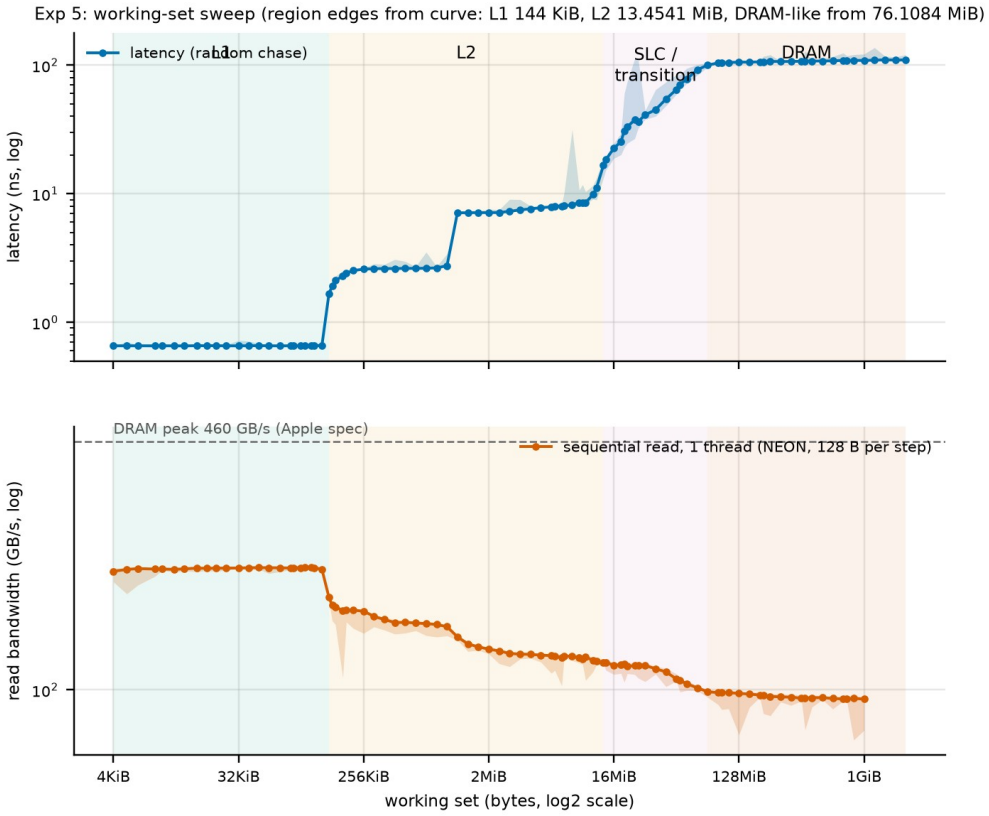


Figure 7. Working-set sweep with annotated regions: latency (top, random chase, median of 12 rounds, dots: rounds) and one-core sequential read bandwidth (bottom, n = 9) versus footprint. Shaded regions: L1, L2, SLC transition and DRAM, from the curve. Dashed line: 460 GB/s peak. Fixed: one thread, QoS user-interactive. Varied: footprint.

One-core sequential read bandwidth is 211 GB/s at 32 KiB, 162 GB/s at 256 KiB, 128 GB/s at 2 MiB, 116 GB/s at 16 MiB, 101 GB/s at 64 MiB and 94 GB/s at 1 GiB *(measured)*. The latency edges are at 144 KiB (L1), 13.5 MiB (start of the ramp) and 76 MiB (DRAM plateau), against capacities of 128 KiB and 16 MiB. The L1 edge matches one sweep step. The L2 edge is 16% below the capacity, so H5a holds to within one to two sweep steps and not exactly. L1 bandwidth (211 GB/s) is below the 300 GB/s predicted for the L1 level (H5a refuted for that number). The bandwidth curve falls where the latency curve rises, as predicted.

6.1 The plateau between L1 and L2

The latency between about 144 KiB and 1.1 MiB is 2.6 ns, which neither the 128 KiB L1 nor the 16 MiB L2 explains. A dependent chase that repeats a fixed address sequence could in principle be helped by a load-address or value predictor with a capacity of several thousand entries; 1.1 MiB at 128 B per node is about 9000 nodes. Two tests separate "bytes" from "node count":

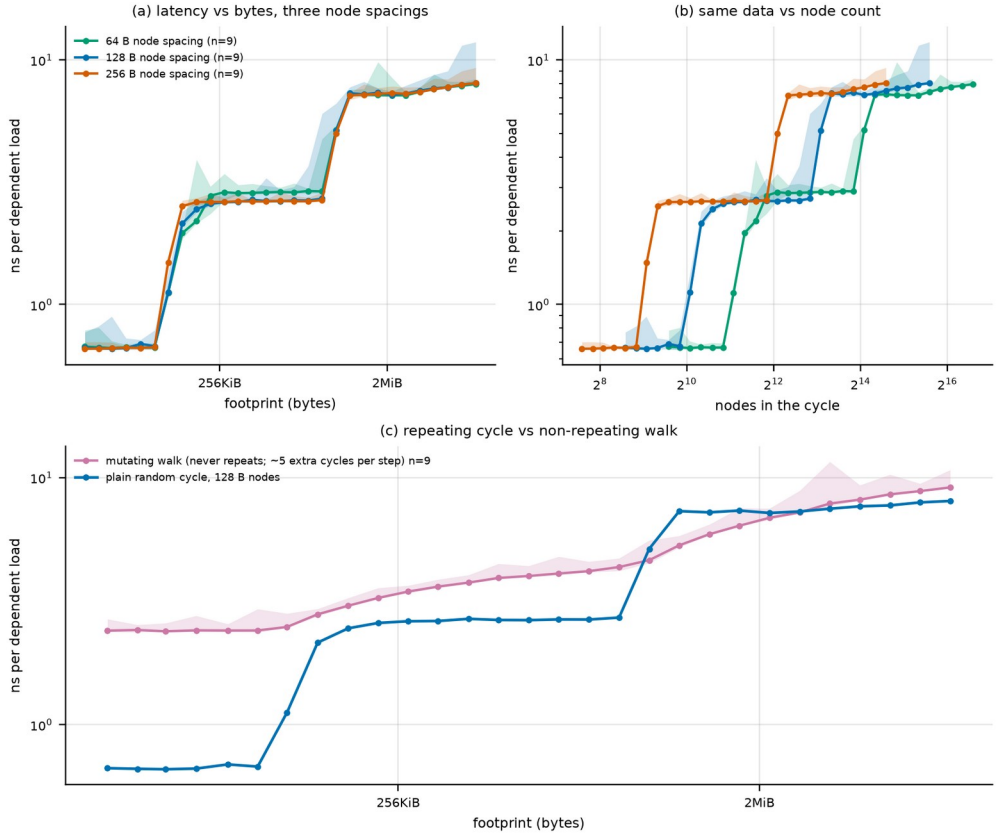


Figure 8. Plateau tests. (a) Latency versus footprint in bytes for node spacings of 64, 128 and 256 B. (b) The same data against the number of nodes. (c) Plain random cycle versus a mutating walk whose next address is a hash of the value just loaded, with a fresh value written at each visit, so the address sequence never repeats. Median and min to max over 9 rounds.

(a) With 64, 128 and 256 B spacing the nodes differ by a factor of four in number at the same footprint. The step up from L1 (135 to 160 KiB) and the step to the L2 level (between 0.9 and 1.3 MiB) are at the same footprints in bytes for all three spacings and not at the same node counts (Figure 8b) *(measured)*. A predictor with a fixed number of node entries is therefore contradicted. (b) The mutating walk shows no flat plateau, but it also differs statistically from the cycle (it visits nodes with replacement, so a cache-like level of fixed size produces a gradual rise of the hit fraction, not a step). Its latency rises smoothly from 2.4 ns at 48 KiB to 4.6 ns at 1.1 MiB. This test cannot separate the explanations and is inconclusive. The plateau is reported as an unexplained cache-like level that follows capacity in bytes *(measured)*; its identity is not established.

7. Experiment 6: effect of cache misses on kernels

Hypotheses: H6a to H6e. Expected: ns per element is flat while the footprint is below L2 and rises beyond it by a factor of 3 to 15; strides raise the cost until the stride reaches a line; a random gather is at least 8 times slower than unit stride in DRAM; an AMAT model built from the measured latency explains the random cases within a factor of two and fails for unit stride where the prefetcher dominates.

Setup. Two f32 kernels, z[i] = x[i]*y[i] and y[i] = a*x[i] + y[i], over arrays from 1 K to 128 M elements (footprints of 8 KiB to 1.5 GiB), each with unit stride, element strides of 2 to 64 (8 B to 256 B) and a scattered index (a Weyl sequence). Miss behavior is inferred from timing. Correctness: both kernels are checked against a scalar reference in the self-test (multiply bit-exact, the second kernel computed without contraction).

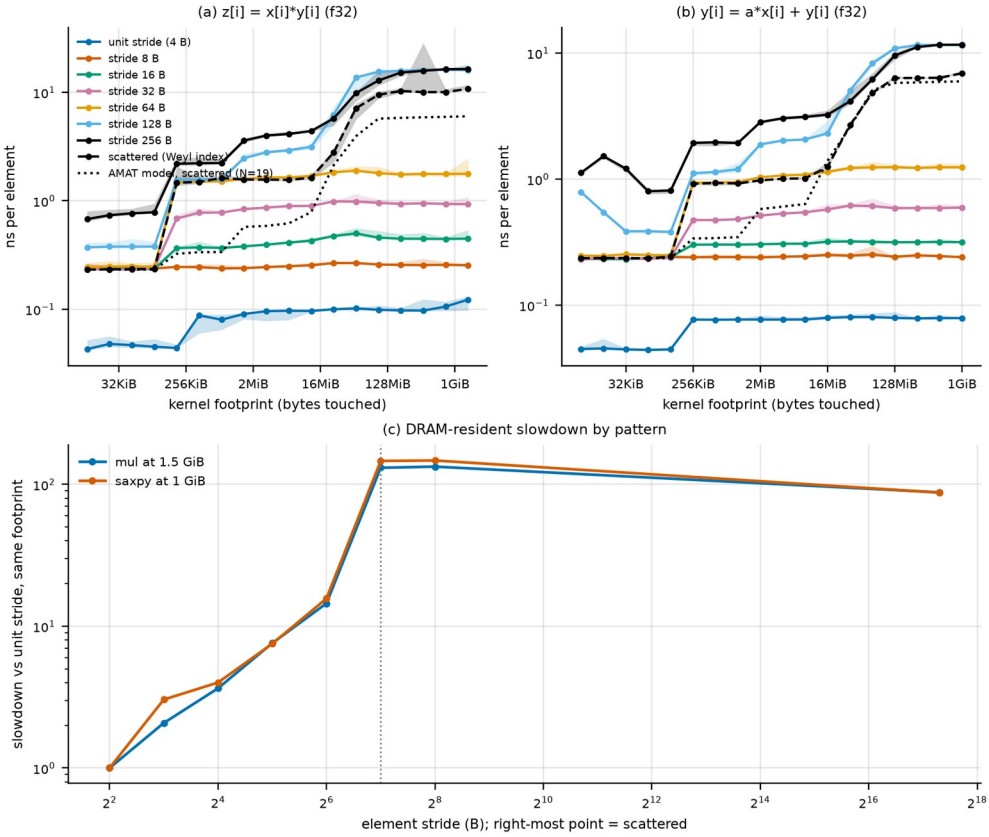


Figure 9. Kernel runtime. (a, b) ns per element versus footprint for unit stride, strides 8 to 256 B and a scattered index, with the AMAT model for the scattered case (dotted). (c) Slowdown versus unit stride at the largest footprint. Median and min to max over 5 rounds. Fixed: one thread. Varied: footprint and index pattern.

Results. Unit-stride cost rises from 0.043 ns per element at 12 KiB to 0.123 ns at 1.5 GiB for z = x*y, 2.9x, and 1.8x for the second kernel, below the 3 to 15 predicted (H6a) because the prefetcher hides the latency. The cost also doubles when the footprint leaves L1 and rises further, more slowly, inside L2. The scattered kernel is 5.4x slower than unit stride at 12 KiB and 88x slower at 1.5 GiB for z = x*y (87x for the second kernel at 1 GiB), satisfying H6c (at least 8x) at the DRAM-sized footprints (and already 16x at 6 MiB). Strides raise the cost sharply up to 128 B, where the cost per element reaches 16 ns, but the cost is not flat beyond the line size (Figure 3b: 4.4 ns at 192 B, 6.5 ns at 320 B, 17 ns at 384 B), so H6b ("flat after the line size") is refuted; the structure follows multiples of 128 B (Section 3.1).

Table 7. AMAT-style prediction: ns/element = t_hit + miss ratio x (L(F) - L_L1) / N, with L(F) the measured chase latency at footprint F and N = 19 lines (Section 5); miss ratio 1/32 for unit stride (32 floats per line), 1/2 for a 64 B stride and 1 for scattered.

kernel	pattern	smallest ns/elem	largest ns/elem	ratio	AMAT prediction at largest, ns	measured / predicted
z = x*y	unit	0.043	0.123	2.9x	0.224	0.55
z = x*y	stride 64 B	0.244	1.775	7.3x	3.135	0.57

kernel	pattern	smallest ns/elem	largest ns/elem	ratio	AMAT prediction at largest, ns	measured / predicted
z = x*y	scattered	0.232	10.766	46.5x	6.013	1.79
y = a*x + y	unit	0.045	0.079	1.8x	0.224	0.35
y = a*x + y	stride 64 B	0.247	1.238	5.0x	3.110	0.40
y = a*x + y	scattered	0.236	6.903	29.3x	5.963	1.16

Interpretation. *(inferred)* The scattered pattern defeats both the cache (each element in a different line) and the prefetcher; its cost at DRAM footprints, 10.8 ns per element, is within a factor of 1.8 (first kernel) and 1.2 (second kernel) of the AMAT model (H6d holds for the random cases). For unit stride the model predicts a cost 1.8 (first kernel) and 2.8 (second kernel) times too high because the prefetcher streams the data; H6d's second part (more than 2x off) holds only for the second kernel. Hardware miss counters were not available in the main dataset, so the miss ratios in the model are assumptions from the footprint and the line size.

8. Experiment 7: TLB-miss impact

Hypotheses: H7a to H7d. Expected: with a fixed number of lines, touching one line per distinct 16 KiB page adds latency beyond the L1 DTLB reach (128 to 512 pages), a larger step at 2048 to 8192 pages, and a penalty of 10 to 30 ns at 65 536 pages. Huge pages cannot be tested.

Setup. A random single-cycle chase over L lines. "Packed": the L lines contiguous (L/128 pages). "Paged": one line in each of L different 16 KiB pages (same cache footprint, L pages). Three placements of the line inside its page were compared because the placement decides which cache sets the lines map to: the original placement (in-page offset equal to the line index modulo 128), uniformly random offsets (a fixed seed), and a constant offset (worst case for set conflicts). A second panel keeps 32 768 lines (4 MiB) fixed and spreads them over 256 to 32 768 pages. The translation penalty is defined as paged with random offsets minus packed. 12 rounds.

Table 8. Median ns per dependent load by placement and pages touched. The penalty column uses random offsets; the "earlier definition" column uses the original placement.

pages (= lines)	packed	original layout	random offsets	constant offset	translation penalty (random - packed)	earlier definition (original - packed)	n
16	0.66	0.66	0.66	2.69	+0.00	+0.00	12
32	0.66	0.66	0.66	2.90	−0.00	−0.00	12
64	0.65	0.66	0.66	3.08	+0.00	+0.00	12
128	0.66	0.66	0.66	6.68	+0.00	+0.00	12
256	0.66	1.97	1.97	8.63	+1.31	+1.31	12
512	0.66	1.97	2.00	8.81	+1.35	+1.31	12
1024	0.66	1.97	2.93	26.31	+2.28	+1.32	12
2048	2.58	7.36	4.01	43.46	+1.42	+4.78	12
4096	2.62	11.99	8.02	46.93	+5.39	+9.37	12
8192	3.35	12.53	10.95	47.50	+7.60	+9.18	12
16 384	7.13	12.94	13.09	69.27	+5.96	+5.81	12
32 768	7.64	18.70	13.62	108.28	+5.98	+11.06	12
65 536	8.11	24.39	15.35	108.92	+7.24	+16.29	12

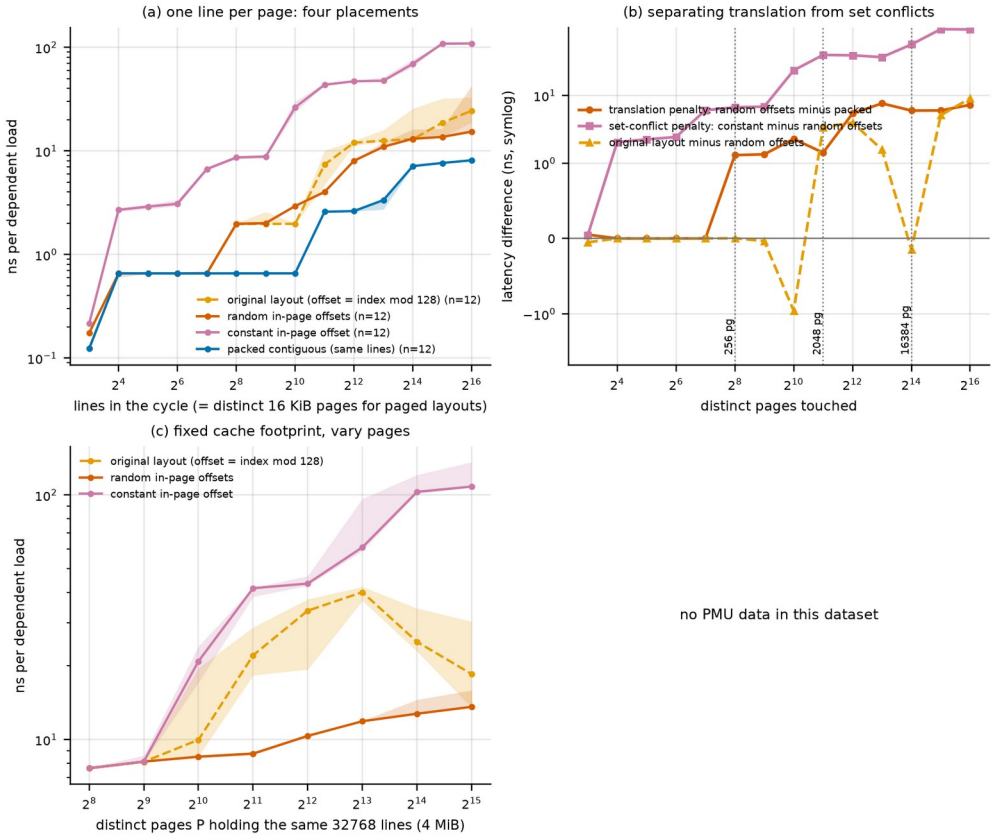


Figure 10. TLB experiment. (a) Latency versus lines (= pages for the paged placements) for four placements. (b) Differences: translation penalty (random offsets minus packed), set-conflict penalty (constant minus random offsets) and the original placement minus random offsets, on a symmetric log axis. (c) 32 768 lines over P pages. (d) PMU counters if present. Median and min to max over 12 rounds. Fixed: one thread, 16 KiB pages. Varied: pages touched and in-page placement.

Old and new results side by side. With the original placement the apparent penalty (paged minus packed) is +1.3 ns at 256 pages, +4.8 ns at 2048, +9.4 ns at 4096, a dip to +5.8 ns at 16 384 and +16.3 ns at 65 536. With random in-page offsets the penalty is +1.3 ns at 256 pages, +5.4 ns at 4096 pages and +7.2 ns at 65 536 pages. The constant offset adds up to 94 ns more at 65 536 pages: with every line at the same in-page offset the lines compete for few cache sets and the latency reaches that of DRAM (109 ns) although the footprint is only 8 MiB. In the fixed-footprint panel the original placement produces a hump (40 ns at 8192 pages, falling to 18.5 ns at 32 768) that random offsets remove (a smooth rise from 7.6 to 13.6 ns). **Conclusions that changed:** the penalty at 65 536 pages is about 7 ns, not 16 ns (below the 10 to 30 ns of H7b); the fall in the fixed-footprint panel was a cache-conflict artifact; the onset at 256 pages (+1.3 ns), the step near 4096 pages and the monotonic rise in the large are unchanged. Small non-monotonic wiggles remain (the penalty at 2048 pages is below that at 1024; at 16 384 below that at 8192), up to 1.6 ns *(measured)*.

Interpretation. *(inferred)* The onset at 256 pages (4 MiB reach) matches a first-level data TLB of about 128 to 256 entries of 16 KiB and the step at 4096 pages (64 MiB) a second-level structure; the exact sizes were not measured and the packed curve itself rises from 2.6 ns at 2048 lines to 8.1 ns at 65 536 lines because the cache footprint (8 MiB) grows. Hardware TLB miss counters would settle the attribution; none were obtained in this dataset. Huge pages: the comparison was not possible (Section 1.3), so no statement about huge pages is made.

9. Experiment 8: benchmark pitfalls

Hypotheses: H8a to H8d. Each pitfall is shown with the misleading result next to the corrected one (Figure 11).

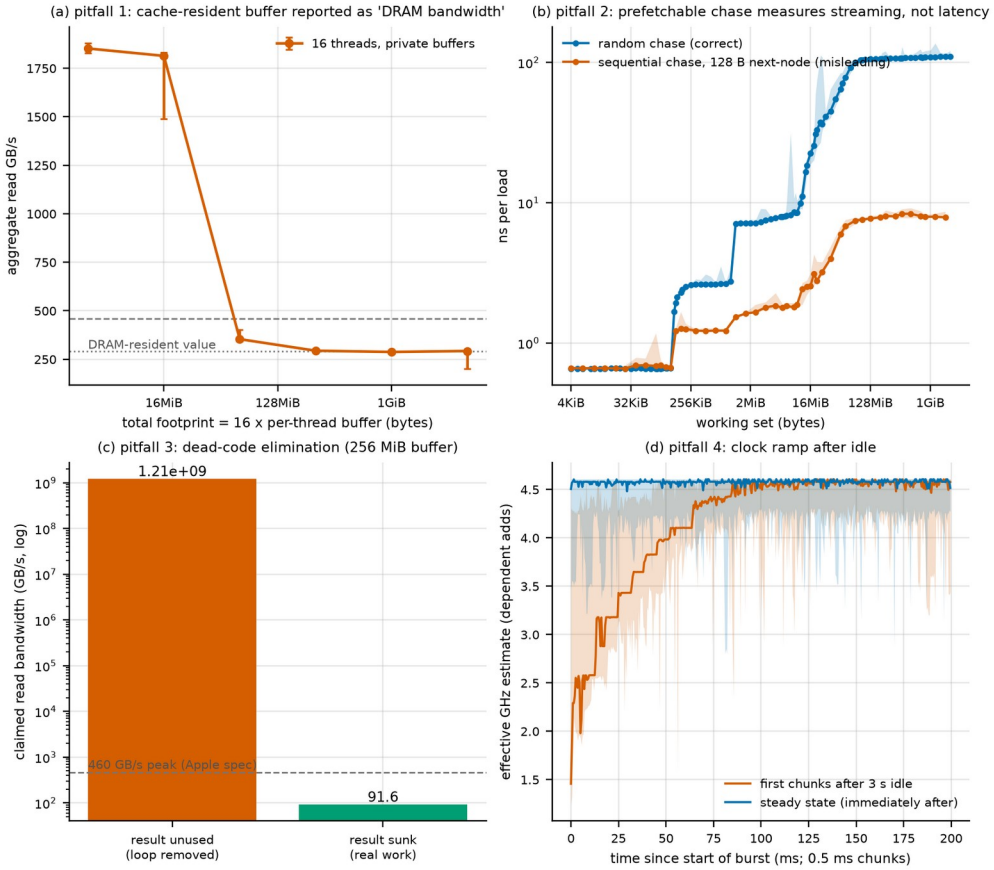


Figure 11. Pitfalls. (a) Aggregate "DRAM bandwidth" of 16 threads versus total buffer size (cache-resident buffers give a misleading value). (b) Sequential versus random chase over the footprint sweep. (c) A kernel whose result is unused versus the same kernel with a sink. (d) Effective clock after 3 s idle versus steady state. Median and min to max over 5 rounds (a, c, d) and 12 rounds (random chase).

1. Cache-resident buffers. A 16-thread read test with 256 KiB per thread (4 MiB total) reports 1852 GB/s; the same test at 256 MiB total reports 294 GB/s, the DRAM value. The misleading value is 6.3x too high and exceeds the 460 GB/s peak, which flags it as impossible for DRAM (H8a).

2. Prefetchable chase. A sequential chase over a 1 GiB footprint reports 7.9 ns per load against 109 ns for the random chase, 14x too low (H8b).

3. Dead-code elimination. A read loop whose result is never used has no symbol in the compiled program (the optimizer deleted it; Appendix E shows the call site: only the sinking variant is called). Timing it claims 1.2e+09 GB/s, against 91.6 GB/s once the result is summed into a sink (H8c).

4. Clock ramp. A process that has been idle for 3 s runs its first 5 ms at 43% of the steady clock, 20 ms at 69%, 50 ms at 87% and reaches the steady 4.58 GHz after about 100 ms. A benchmark that times a short region right after idle underestimates the speed by up to a factor

of two (H8d: the 1.1x slowdown predicted is far exceeded). This is why every process spins for 300 ms first.

10. AMAT and Little's Law synthesis

Average memory access time for a dependent load stream is the hit-weighted sum of the level latencies of Section 2: 0.66 ns (L1), about 7.1 ns (L2), 92 ns in the system-level-cache ramp at 64 MiB and 109 ns (DRAM) *(measured)*. For independent accesses the average cost per access divides the miss penalty by the number of misses in flight, which is Little's Law, N = bandwidth times latency divided by the line size:

Table 9. Little's Law across the experiments. For rows marked inferred N is computed from the measured bandwidth and latency; no independent count of in-flight requests is available.

Case	Latency	Lines in flight N	Bandwidth = N x 128 B / latency	Source
One core, random demand misses	110 ns	18.9	21.9 GB/s	Exp 4a; N is known by construction up to the plateau
One core, sequential stream	121 ns	83 (inferred)	88 GB/s	Exp 4b; prefetcher raises N
Chip at the knee (5 threads)	149 ns	333 (inferred)	287 GB/s	Exp 4b
Chip, 16 threads	201 ns	453 (inferred)	289 GB/s	Exp 4b; flat bandwidth, longer queue

The AMAT-style model of Exp 6 (Table 7) reproduces the scattered kernel within a factor of 1.8 using only the measured latency curve and the one-core concurrency of 19 lines, and over-predicts the unit-stride cost where the prefetcher raises the concurrency far above that value. The picture that fits all experiments *(inferred)*: latency is set by the level that serves the load; one core's throughput is its number of outstanding misses divided by that latency (about 19 for demand misses, many more with prefetching); the chip's throughput saturates near 293 GB/s with 5 to 6 streaming cores, after which extra requests only lengthen the queue and the latency; and which internal limit sets that plateau is not determined.

11. Anomalies and unresolved items

Tested, and the test narrowed the explanation

1. *Read-only ceiling.* Streaming reads saturate at 293 GB/s (64% of peak) while read-modify-write reaches 374 GB/s and read plus stnp writer threads move 355 GB/s in total. Excluded by test: the load instruction mix, the buffer content, the DRAM peak. Not excluded and untested: limits on read-return bandwidth, on outstanding requests, or QoS-related throttling (background threads reach only 46 GB/s). Cause: unknown.
2. *The 2.6 ns plateau between L1 and L2.* Follows bytes, not node count (a node-count predictor is excluded). Its identity is not established; the non-repeating-walk test was inconclusive.
3. *Slow strides.* The penalty at 128, 256, 384, 512, 640 and 1024 B strides follows multiples of the 128 B line, not powers of two; a hashed offset does not remove it. Why 192 B and 320 B strides are faster is untested.
4. *Content-dependent stores.* Zero lines and uniform lines are written faster than random lines. The mechanism (detection of uniform lines, compression, write combining) is untested.
5. *TLB placement artifact.* Explained: the original placement mapped lines to a conflicting set of cache sets in the fixed-footprint panel; random offsets remove it.

Unexplained and untested

6. Random 128 B independent loads deliver about a third of the line traffic of random 16 B loads (7.5 GB/s against 22.7 GB/s of line-equivalent traffic) in the same kernel.
7. The sequential chase is faster at a 1 KiB stride (6.8 ns) than at 256 B and 512 B (12.0 and 14.3 ns).
8. The latency ramp toward the system-level cache begins at 13.5 MiB, below the 16 MiB capacity.
9. Small non-monotonic wiggles in the TLB penalty (up to 1.6 ns) and a cycle of exactly 8 nodes that reads 0.12 ns per load, below the L1 latency (the loop unroll repeats the same 8 addresses; excluded from all results as degenerate).

10. The probe latency at 16 threads occasionally shows rounds of tens of microseconds (visible as the upper bars of Figure 4d), and the 70/30 plain-store mix at 16 threads has a median probe latency of 549 ns. Background activity cannot be excluded; medians are reported.
11. Which cores run the bandwidth threads above 6 threads cannot be observed.

12. Limitations

- **No hardware counters in the main dataset** (Section 1.3, Appendix H). Every cache and TLB statement is inferred from timing.
- **The clock is estimated** from a dependent-add chain assuming one cycle per add. powermetrics was not run: it requires administrator rights and the password is not available to the assistant. A script that runs it during a single-thread spin was prepared. **Frequency cross-check: not done** (open item). All cycle values remain estimates based on the dependent-add calibration.
- **The peak, 460 GB/s, is a specification**, not a measurement; the validation flags only values above it.
- **No pinning.** Threads run at user-interactive QoS; the Super-core signature is verified for the first thread only.
- **No huge pages**, no SLC capacity, no fixed frequency.
- **Single machine, single operating-system image, battery power** for the main collection (an earlier AC collection agreed within 1%). Background daemons were active for a few percent of one core; the medians over 5 to 12 rounds absorb them, the minimum-to-maximum ranges show them.
- **Rounds:** n is 12 for the headline experiments and 5 to 9 elsewhere; with n below 10 only minimum-to-maximum ranges are shown.
- **The AMAT model** uses assumed miss ratios (no counters).

13. Benchmark flaws found and fixed

Each flaw was found by inspecting a result that looked implausible; the code was corrected and the dependent data re-collected.

1. **Dependency chain in the read kernels.** One XOR accumulator capped one-thread read bandwidth at about 73 GB/s at every footprint. Independent accumulators removed it (now 94 GB/s at 1 GiB, 211 GB/s at 32 KiB).
2. **Stride sweeps shrank the cache footprint.** A fixed byte footprint reduced the distinct lines at large strides (a random chase at 16 KiB stride read 29 ns instead of about 109 ns). The sweeps now hold 1 Mi distinct lines.
3. **The latency probe lowered its own clock** by sleeping 200 ms before measuring (idle 119 ns against 109 ns). It now spins.
4. **The injection-delay range was too weak** to reach low loads; it now reaches 3.2 million loop iterations.
5. **A degenerate 8-node cycle** read 0.12 ns per load, below the L1 latency; the sweep starts at 2 pages and the point is excluded.
6. **Store kernels wrote uniform lines**, which made stnp and dc zva look twice as fast as plain stores. With random data the stnp advantage is 33%. Zero and uniform variants are kept as a labeled test (Table 5).
7. **The TLB layout mixed cache-set conflicts with translation cost** (Section 8); random offsets and a constant-offset control separate them.
8. **The knee was first computed by ordering points by bandwidth**, which selected 10 threads on this noisy curve. The definition as written (first crossing along the thread sweep) selects 5 and is reported.
9. **Numbers typed into text by hand disagreed with the data tables in places.** All numbers here are generated from the processed data by a script and recomputed from the raw files by a second script.

Appendix A. Code map

File (Project-2 folder)	Purpose
src/membench.c	All benchmark kernels and sub-commands: chase, tlb, bw, pat, mlp, loaded, kern, dce, ramp, mut, hugetest, selftest
src/common.h	Timer, page-aligned mapping, QoS helpers, argument parsing, long-format CSV output
src/pmu.h	Optional PMU access through the private kperf frameworks (needs root)
scripts/build.sh	Compiles with the flags listed in Table 2
scripts/run_exp.py	Experiment definitions; shuffled rounds; machine-readable metadata per run
scripts/meta.py	Hardware, OS, compiler, power, thermal and policy metadata
scripts/parse.py, scripts/validate.py	Median, min, max, n per configuration; sanity checks
scripts/plot.py, scripts/analysis_lib.py	Figures and tables

File (Project-2 folder)	Purpose
scripts/build_report.py, scripts/check_report.py	Assemble this report from the data; recompute its numbers from the raw files
scripts/run_privileged.sh	powermetrics and PMU counter run (needs sudo)

Appendix B. Deviations from the assignment

Table 10. Deviations from the assignment.

Assignment asks for	What was done instead	Consequence
Intel Memory Latency Checker	Own benchmarks: random-cycle pointer chase, NEON streaming kernels, loaded-latency harness (probe chase thread plus N bandwidth threads with adjustable injection delay)	Absolute numbers are not comparable to MLC output; the method is described in Section 1.1
perf cache and TLB miss counters	Counters through Apple's private kperf interface need administrator rights. See Appendix H for the attempts and outcome	No counter values; all cache and TLB statements are inferred from timing (labeled as such)
Thread pinning, governor, SMT off	Not available on macOS (no taskset, no governor, no SMT on this chip). QoS class set per thread; DVFS left automatic and recorded	Placement is inferred, not guaranteed; frequency is estimated, not fixed
Huge-page comparison	Not possible: mach_vm_allocate with VM_FLAGS_SUPERPAGE_SIZE_2MB and _ANY, and mmap with the superpage flag, all fail with "invalid argument"	TLB effects are shown through page locality only (Section 8)
L3 cache	The chip has a system-level cache whose size sysctl does not report	Shown as a latency ramp between L2 and DRAM; no capacity is claimed
Frequency in cycles	Cycles are ns times an estimated clock	Every cycle value is labeled an estimate
NUMA	Unified memory, no NUMA	not applicable

Reproduction.

From the Project-2 folder: scripts/run_all.sh rebuilds the program, runs the self-test, parses the stored raw data, validates and plots. COLLECT=full scripts/run_all.sh re-measures everything (about 30 minutes on a quiet machine, AC power). sudo scripts/run_privileged.sh adds the counter run and the powermetrics check.

Appendix C. Hypotheses, measured results and verdicts

Predictions were written before the benchmarks existed (the first group, H1a to H8d) or before the corresponding new measurements were read (the second group, H7d to H13, Appendix D). Text is reproduced without change except that references to an unrelated earlier project were removed.

Table 11. Predictions and verdicts. HV1 to HV3 are validity checks. "Partly" means that part of the prediction held.

ID	Prediction (written before the measurement)	Measured result	Verdict
HV1	A worker thread that sets QOS_CLASS_USER_INTERACTIVE on itself runs on a Super core. Test: the random-chase latency curve has its L1-to-L2 step at 128 KB (not 64 KB). Refuted if the step sits at 64 KB.	The L1 to L2 step is at 144 KiB (sweep step above 128 KiB); user-interactive threads show Super-core signatures (Section 1.5).	**Confirmed**
HV2	Repetition noise (IQR/median) is at most 5% for L1, L2 and DRAM plateau points and may reach 15% in the SLC ramp.	Interquartile range over median: 0.4% (L1), 0.4% (L2), 1.6% (DRAM). In the system-level-cache ramp the range over 12 rounds at 64 MiB is 84 to 101 ns (median 92).	**Confirmed for plateaus; ramp within about 15% of the median**
HV3	Every raw bandwidth value is below 460 GB/s (peak for the 32-core-GPU M5 Max). A value above 460 GB/s in a footprint beyond the SLC	No DRAM-resident bandwidth above 460 GB/s; maximum 293 GB/s read and 277 GB/s for zero-data stores (flagged as not DRAM	**Confirmed**

ID	Prediction (written before the measurement)	Measured result	Verdict
	is a measurement error and will be investigated; above it with a cache-resident footprint is expected.	bandwidth).	
H1a	L1 hit latency 3 to 5 cycles, i.e. 0.6 to 1.2 ns.	0.66 ns, 3.0 cycles (estimated).	**Confirmed**
H1b	L2 latency 12 to 40 cycles, i.e. 3 to 9 ns; the plateau sits near 16 MB capacity (one active core can use the whole 16 MB L2).6 ns step between 128 KB and 1 MB; I predict that step recurs.	L2 7.1 ns (33 cycles, estimated) at 2 MiB; plateau of 2.6 ns between 144 KiB and 1.1 MiB recurs; the ramp toward SLC starts at 13.5 MiB, not at 16 MiB.	**Latency confirmed; capacity prediction partly (13.5 MiB)**
H1c	The SLC region (16 MB up to some hundreds of MB) is a ramp, not a clean plateau, with values between L2 and DRAM. An SLC size cannot be read exactly; I predict the ramp ends before 512 MB.	Ramp from 13.5 MiB to 76 MiB, no plateau; DRAM level reached before 512 MiB.	**Confirmed**
H1d	DRAM latency (1 GiB footprint, random lines) is 90 to 130 ns, 400 to 600 cycles.	108.7 ns, 498 cycles (estimated).	**Confirmed**
H1e	Latency is monotonic non-decreasing with working set within noise (5%).	No violation of monotonicity within 5%.	**Confirmed**
H2a	Sequential pointer-chase at 128 B stride in a DRAM-resident footprint is at least 5x faster per node than the random chase (prefetcher;	Sequential chase 7.8 ns against 108.4 ns: 13.9x.	**Confirmed**
H2b	Random chase latency does not depend on node stride for strides of 128 B and up (flat within 10% once the footprint is fixed at 1 GiB), because each node is a different line anyway. TLB effects raise the large-stride points modestly.	Random chase 108 to 114 ns for 64 B to 4 KiB strides (1 Mi lines, amended design).	**Confirmed (amended)**
H2c	A sequential chase with growing stride loses its prefetch benefit gradually; at 16 KB stride (one node per page) it is still at least 2x faster than random.	Sequential chase at 2 KiB and 4 KiB strides: 103 and 111 ns, no benefit; benefit ends between 1 and 2 KiB. 16 KiB was not tested.	**Refuted within the tested range (amended)**
H2d	Single-thread sequential read bandwidth in DRAM is 60 to 140 GB/s; random-line (independent loads) bandwidth is 20 to 70% of sequential.	One-core sequential DRAM read 94 GB/s (in range). Random-line rate 8% of sequential for independent 128 B loads (below the range) and 23% for the random-chain test.	**Partly**
H2e	Useful-byte bandwidth for a 16 B access at stride 128 B or larger is about 1/8 of the full-line rate (the 128 B line is fetched whole); stride 64 B behaves like stride 128 B for line-count purposes (two accesses per line).	A 16 B access at 128 B stride gives 1/33 (sequential) and 1/2.6 (random) of the whole-line rate.	**Refuted**
H3a	Single-thread throughput in a DRAM-resident footprint orders as 100% read >= 70/30 >= 50/50 >= 100% write in application-visible GB/s, with 100% write between 0.4x and 1.0x of read.	100% read 87, 70/30 101, 50/50 108, 100% write 68 GB/s: mixed traffic beats read-only; write/read = 0.78.	**Refuted (ordering); write ratio confirmed**
H3b	With write-allocate, 100% write moves more DRAM bytes than the application writes; a traffic-corrected rate (writes counted twice) for 100% write is closer to the read rate than the uncorrected one. This is a model, not a measurement (no DRAM counters).	The modeled DRAM rate of 100% write (136 GB/s) is farther from the read rate than the uncorrected 68 GB/s. The model is not testable without DRAM counters.	**Refuted as phrased; write-allocate not established**
H3c	Non-temporal stores (stnp) change write throughput by less than 15% (the hint may be ignored in cached hierarchies). dc zva (zero a line without reading it) is at least 1.3x faster than plain stores if user-mode DC ZVA is permitted, which would be direct evidence of write-allocate cost.	stnp with random data is 33% faster (not within 15%); dc zva is 4.0x faster than plain stores of random data but equals plain stores of zero data (277 GB/s).	**Refuted (stnp); zva speed confirmed but not evidence of write-allocate**
H3d	Under a fixed background load, a write-heavy mix raises loaded read latency more than a read-only mix of the same application-visible rate.	Not tested at matched application rates. At 16 threads write-heavy mixes show higher probe latency per GB/s than read-only.	**Not tested**

ID	Prediction (written before the measurement)	Measured result	Verdict
H4a	One thread running K independent random chains: throughput rises roughly linearly with K up to a plateau between K = 16 and 64 outstanding lines; the plateau throughput is 20 to 60 GB/s. At K = 1 the rate is 128 B / latency, about 1.2 GB/s.	Plateau 21.9 GB/s from K about 20 (range 16 to 64, 20 to 60 GB/s); K = 1: 1.2 GB/s.	**Confirmed**
H4b	Multi-thread streaming reads: aggregate bandwidth saturates at 55% to 85% of 460 GB/s (250 to 390 GB/s) with 8 to 14 threads; adding threads beyond saturation does not raise throughput.	Saturation at 64% of 460 GB/s, reached with 5 to 6 threads.	**Partly (range yes, 8 to 14 threads no)**
H4c	Loaded latency measured by an independent probe thread stays within 1.3x of idle latency up to about 50 to 60% of the saturation bandwidth, then rises, reaching at least 2x idle at saturation (the knee).	Latency 1.26x idle at 4 threads (93% of maximum bandwidth) and 1.35x at the maximum; 2x only at 14 threads.	**Refuted as phrased**
H4d	Little's Law check: in-flight lines N = BW x L_loaded / 128 B is system-wide 200 to 700 lines at the knee and does not keep growing past it; BW grows at constant N only if latency is constant, so beyond the knee BW stays flat while L rises.	N = 333 lines at K1; N keeps rising to 453 at 16 threads while bandwidth is flat.	**Partly**
H4e	More than 6 bandwidth threads spill onto Performance cores and the per-thread rate falls.	Per-thread rate falls with threads, but saturation alone explains it; placement above 6 threads is not observable.	**Not separable**
H5a	L1 edge at 128 KB (+/- one sweep step), L2 edge at 16 MB (+/- one step), DRAM plateau by 512 MB. The same sweep for single-thread read bandwidth falls from above 300 GB/s (L1) to 100 GB/s class (DRAM). The bandwidth edge locations match the latency edges.	L1 edge at 144 KiB (128 KiB expected), ramp from 13.5 MiB (16 expected), DRAM plateau from 76 MiB; one-core L1 bandwidth 211 GB/s (not above 300).	**Partly**
H6a	For z[i] = x[i]*y[i] and y[i] = a*x[i] + y[i] (f32, unit stride), ns per element is flat while the footprint is below the L2 size and rises beyond it; the rise ratio between L1-resident and DRAM-resident is 3x to 15x.	Unit-stride rise 2.9x (first kernel) and 1.8x (second); cost not flat inside L2.	**Refuted**
H6b	Stride-k element access (k x 4 B) raises ns per element until k x 4 B reaches 128 B (k = 32), after which it is flat; the plateau is at least 5x the unit-stride DRAM cost.	Cost rises to 128 B strides but is not flat beyond (192 B: 4.4, 384 B: 17.2 ns per element).	**Partly**
H6c	A random gather (permuted index) in DRAM is at least 8x slower per element than unit stride.	Scattered 88x unit stride at 1.5 GiB (5.4x at 12 KiB).	**Confirmed at DRAM footprints**
H6d	An AMAT-style prediction built only from the measured latency curve (miss ratio from the footprint/line model, hit and miss latencies from Exp 1, divided by the measured MLP) explains runtime within a factor of 2 for the random cases and fails (under-predicts speed by more than 2x) for unit stride where the prefetcher dominates. No hardware counter is used; miss behavior is inferred.	Measured/predicted: scattered 1.8 and 1.2; unit stride 0.55 and 0.35.	**Partly**
H6e	Both kernels match a straightforward scalar reference bit-for-bit for multiply and within 1 ulp for the fused form.	Self-test: both kernels match the scalar reference.	**Confirmed**
H7a	Holding the number of lines fixed (cache footprint identical), touching one line per page instead of packed lines adds latency once the number of distinct 16 KB pages exceeds the L1 DTLB reach. Onset between 128 and 512 pages (2 to 8 MB reach);3 ns at 256 pages.	Translation penalty (random offsets minus packed) +1.31 ns at 256 pages, none at 128.	**Confirmed**
H7b	A second, larger step occurs between 2048 and 8192 pages (L2 TLB reach of 32 to 128	Step between 2048 and 4096 pages; penalty at 65 536 pages +7.2 ns (below 10 to 30 ns); small	**Partly (magnitude below range)**

ID	Prediction (written before the measurement)	Measured result	Verdict
	MB), and the penalty at 65 536 pages is 10 to 30 ns. Penalty is non-decreasing in pages.	non-monotonic wiggles up to 1.6 ns.	
H7c	Huge-page comparison cannot be done on macOS; no claim about it will be made.	Not attempted: superpage allocations fail on this system.	**Not possible**
H8a	A "DRAM bandwidth" test with an 8 MB array (L2-resident) reports at least 2x the true DRAM bandwidth.	4 MiB total: 1852 GB/s against 294 GB/s DRAM, 6.3x.	**Confirmed**
H8b	A sequential (non-random) pointer chase of a 1 GiB footprint reports at least 5x lower latency than the random chase.	14x lower latency for a sequential chase.	**Confirmed**
H8c	A kernel whose result is never used is removed by -O2: it reports a time near zero or a bandwidth above 1 TB/s. Adding a sink restores a physically plausible value.	Function removed from the binary; claims 1.2e+09 GB/s against 91.6 GB/s with a sink.	**Confirmed**
H8d	After 2 s of idle, the first 10 ms of a compute loop is at least 1.1x slower than steady state (DVFS ramp). This is the least certain prediction; M-series ramp is fast and the effect may be absent. If absent, it is reported as a negative result.	First 5 ms at 43% of steady clock.	**Confirmed (effect far larger than 1.1x)**
H7d	(cache-set aliasing in Exp 7) The original paged layout (in-page offset equal to the line index modulo 128) is neither the best nor the worst placement. A layout with a constant in-page offset has the highest latency at 1024 pages and above (cache-set conflicts on top of translation misses). A layout with uniformly random in-page offsets has the lowest. The translation penalty is redefined as paged_rand minus packed. Prediction: that penalty still starts between 256 and 1024 pages and is non-decreasing, and the dip near 16 384 pages in the original curve is smaller or absent with random offsets. Smoke tests already showed the ordering const > original > random at 4096 pages.	The constant-offset layout is slowest at every size from 128 pages up (109 ns against 15.3 ns at 65 536 pages). The original layout is above random offsets from 2048 pages on, equal at 16 384, and below at 1024 (1.97 against 2.93 ns). The dip at 16 384 pages is smaller with random offsets, not absent; the hump in the fixed-footprint panel disappears.	**Partly confirmed**
H2f	(aliasing in strides) The 128 B and 256 B stride penalty is not specific to power-of-two strides: independent random 16 B loads at 192, 320, 384 and 640 B strides land within 20% of the useful bandwidth of their power-of-two neighbours, and a hashed in-block line offset changes power-of-two strides by less than 20%. Confidence is low.	192 B and 320 B strides are 1.7x faster than 128 B and 256 B although not powers of two; 384 B and 640 B are as slow as 128 B. A hashed line offset changes random 16 B loads by −0%, 3% and 10% (256, 512, 1024 B) and the kernels by 5 to 14% at the largest footprints.	**Refuted for the non-power-of-two part; confirmed for the offset part**
H3e	(content dependence of stores) Store throughput depends on the data. Plain stores of zero lines are at least 2x faster than plain stores of random lines; stnp of random lines is within 1.5x of plain stores of random lines. Consequently the earlier stnp advantage (uniform, non-zero content per line) is not evidence of write-allocate traffic unless it persists with random content. Single-thread read bandwidth is independent of buffer content within 5%.	Zero data 4.1x faster than random for plain stores; stnp random 1.33x plain random; read bandwidth with random buffers within 4% of constant buffers.	**Confirmed**
H4f	(knee) The knee is defined before analysis as (K1) the lowest-load point whose median bandwidth reaches 95% of the maximum median bandwidth measured in the series, and (K2) the point with maximum bandwidth divided by loaded latency. Prediction: K1 falls at 4 to 6 bandwidth threads; K2 is at an equal or lower load than K1; the two differ by at most 2 threads. Little's Law check at every	K1 = 5 threads (within 4 to 6), K2 = 4 threads (at most K1, differing by one thread). In-flight lines per thread are 83 to 28, not within 30% of the one-core plateau of 19; the delay series decreases.	**Partly**

ID	Prediction (written before the measurement)	Measured result	Verdict
	point: lines in flight per thread, N/threads, stays within 30% of the single-core plateau for the thread-count series and decreases for the injection-delay series.		
H11	(read-only ceiling) The plateau of about 300 GB/s for read-only streaming is not caused by the load instruction mix: ldp q-pairs and ld1 four-register loads reach within 10% of the standard kernel at 16 threads. Background QoS threads reach a lower maximum than interactive threads. Adding two concurrent non-temporal writer threads raises the total (read plus write) traffic above the read-only plateau. These tests can exclude causes; they will not establish one.	ldp and ld1 four-register kernels reach 292 to 301 GB/s against 289 to 293 (within 10%); background-QoS threads reach at most 46 GB/s; two stnp writers raise total traffic to 336 to 355 GB/s, two plain-store writers do not (261 to 282 GB/s).	**Confirmed (causes excluded, none identified)**
H12	(2.6 ns plateau) The step between about 144 KiB and 1.1 MiB follows bytes, not node count: with 64 B, 128 B and 256 B node spacing the plateau ends at about the same footprint in bytes. The mutating walk keeps a plateau (a latency increase over L1 of at least 1 ns between 256 KiB and 1 MiB), which excludes a predictor of the repeating address sequence as the sole cause. If both hold the plateau is reported as an unexplained cache-like level.	Both plateau edges are at the same footprints in bytes for 64, 128 and 256 B node spacing. The mutating walk is 1.1 ns above its L1 value at 271 KiB and rises smoothly instead of forming a plateau.	**Bytes confirmed; predictor test inconclusive**
H9	(frequency cross-check) powermetrics reports a single-thread Super-core frequency within 5% of the dependent-add estimate of about 4.55 GHz. Not blind (the estimate is known).	powermetrics not run (needs administrator rights); no cross-check value.	**Open (not run)**
H13	(counters) If PMU access works, L1D load-miss and L2 TLB miss counts per access follow the timing inference: L1D misses per load rise from about 0 below 128 KiB to about 1 beyond 12 MiB for the random chase, and L2 TLB misses per load stay near 0 for packed layouts and rise with pages for paged layouts beyond about 2048 pages.	Counters not read: the kperf interface needs administrator rights. See Appendix H.	**Open (not run)**

Appendix D. Hypotheses amended after seeing data

The original predictions stay visible in Table 11. The table lists every hypothesis whose wording, design or evaluation changed after data were seen, and every prediction added after a first look at results.

Table 12. Amendments.

ID	Original text	Amended text	Reason
H2b	Random chase latency does not depend on node stride once the footprint is fixed at 1 GiB	Footprint is the larger of 1 GiB and 1 Mi lines times the stride (1 Mi distinct lines at every stride)	Holding the byte footprint fixed shrank the number of distinct lines at large strides and made them cache resident (29 ns instead of 109 ns at 16 KiB)
H2c	At 16 KiB stride a sequential chase is still at least 2x faster than random	Evaluated only for strides up to 4 KiB; 16 KiB dropped	The 1 Mi-line design would need 16 GiB at 16 KiB stride
H2d, H3a, H3b, H3c, H5a	Evaluated on the first collection	Evaluated only on data collected after the read-kernel dependency chain was removed	The first read kernels were capped at 73 GB/s at every footprint (Section 13)
H3b, H3c	dc zva and stnp advantages interpreted as evidence of write-allocate traffic	Interpretation withdrawn; random-data variants are the reference and zero and uniform variants are a separate content test (H3e)	Zero and uniform lines run 2 to 4 times faster than random lines (Table 5)

ID	Original text	Amended text	Reason
H4c, H4d (idle latency)	Probe idle latency taken as about 109 ns	Probe warm-up changed from sleeping to spinning	The probe slept and lowered its own clock (119 ns instead of 109 ns)
H7a, H7b	Penalty defined as one line per page minus the same lines packed	Penalty defined as random in-page offsets minus packed	The original placement mixed cache-set conflicts with translation cost
Exp 7 design	Sweep from one page	Sweep from 2 pages	A cycle of 8 nodes reads 0.12 ns, below L1 latency
HV3, H4b (peak)	460 GB/s from secondary sources	460 GB/s from Apple's specification for the 32-core-GPU configuration	Source checked; no percentage changed
H7d, H2f, H3e, H4f, H11, H12, H9, H13	not present	Added with predictions written before the new measurements were read	New tests added to separate confounds (TLB placement, stride aliasing, content dependence, knee definitions, read ceiling, plateau, clock, counters). A few single-run smoke tests were seen first and are disclosed in the prediction text
H4f (K1 implementation)	K1 = lowest-load point reaching 95% of maximum	Unchanged definition; first implementation ordered points by bandwidth (selected 10 threads) and was corrected to first crossing along the thread sweep (5 threads)	Implementation error found when the result disagreed with the thread table

Appendix E. Code excerpts

Pointer chase (one random cycle through all nodes; every load depends on the previous one):

```
static void chase_link(chase_t *c, void **addrs, size_t n, int do_shuffle, uint64_t seed) {
    if (do_shuffle) shuffle_ptrs(addrs, n, seed);
    for (size_t i = 0; i < n; i++) *(void **)addrs[i] = addrs[i + 1 == n ? 0 : i + 1];
    c->cur = addrs[0]; c->n = n;
}
// Node j of a strided layout. For strides above one line the pointer slot moves by whole lines inside the
// stride block so that nodes spread over all cache sets (stride 16 KB would otherwise alias to a few sets).

static __attribute__((noinline)) void *chase_loop(void *p, size_t n) {
    for (size_t i = 0; i < n; i += 8) {
        p = *(void **)p; p = *(void **)p; p = *(void **)p; p = *(void **)p;
        p = *(void **)p; p = *(void **)p; p = *(void **)p; p = *(void **)p;
    }
    return p;
}
```

Streaming bandwidth kernel (independent accumulators; the sum is returned and sunk):

```
static inline uint32x4_t rd_line(const uint8_t *p) {
    const uint32_t *q = (const uint32_t *)p;
    uint32x4_t a = veorq_u32(vld1q_u32(q), vld1q_u32(q + 4));
    uint32x4_t b = veorq_u32(vld1q_u32(q + 8), vld1q_u32(q + 12));
    uint32x4_t c = veorq_u32(vld1q_u32(q + 16), vld1q_u32(q + 20));
    uint32x4_t d = veorq_u32(vld1q_u32(q + 24), vld1q_u32(q + 28));
    return veorq_u32(veorq_u32(a, b), veorq_u32(c, d));
}

#define DEF_MIX(NAME, RL, WL, WR)                                                  \
    static __attribute__((noinline)) uint64_t NAME(uint8_t *A, uint8_t *B, size_t groups) {  \
        uint32x4_t accs[4] = {vdupq_n_u32(0), vdupq_n_u32(0), vdupq_n_u32(0), vdupq_n_u32(0)};  \
        for (size_t g = 0; g < groups; g++) {                                      \
            LOOP_UNROLL for (int k = 0; k < RL; k++) accs[k & 3] = veorq_u32(accs[k & 3], rd_line(A + ((g * RL + k) << 7))); \
            LOOP_UNROLL for (int k = 0; k < WL; k++)                               \
                WR(B + ((g * WL + k) << 7), vdupq_n_u32((uint32_t)(g * 16 + k)));  \
        }                                                                          \
        uint32x4_t acc = veorq_u32(veorq_u32(accs[0], accs[1]), veorq_u32(accs[2], accs[3]));  \
        return vgetq_lane_u32(acc, 0) ^ vgetq_lane_u32(acc, 1) ^ vgetq_lane_u32(acc, 2) ^ vgetq_lane_u32(acc, 3); \
    }
```

Loaded-latency harness (bandwidth threads stream while the probe thread chases; per-thread byte counters give bandwidth):

```
static void *bw_thread(void *arg) {
    bwthr *t = arg; set_thread_qos(t->qos); t->qos_seen = get_thread_qos();
    __atomic_add_fetch(t->ready, 1, __ATOMIC_SEQ_CST);
    while (!__atomic_load_n(t->go, __ATOMIC_ACQUIRE)) __asm__ volatile("yield");
    const size_t slice = (1u << 20) / (10 * LINE); size_t g = 0; uint64_t s = 0;
    const size_t per_group = (size_t)(t->m->rl + t->m->wl) * LINE;
    while (!__atomic_load_n(t->stop, __ATOMIC_RELAXED)) {
        size_t sl = t->groups - g < slice ? t->groups - g : slice;
        s += t->m->fn(t->A ? t->A + g * t->m->a_lines * LINE : NULL, t->B ? t->B + g * t->m->b_lines * LINE : NULL, sl);
        __atomic_store_n(&t->bytes, t->bytes + sl * per_group, __ATOMIC_RELAXED);
        g += sl; if (g >= t->groups) g = 0;
        if (t->delay) { uint64_t x = 0; for (size_t i = 0; i < t->delay; i++) __asm__ volatile("add %0, %0, #1" : "+r"(x)); }
    }
    sink_u64(s); return NULL;

        warm_spin(200);   // keep the probe core busy while the bandwidth threads ramp (an idle sleep here lowered its clock; see report)
        uint64_t b0[32]; for (int i = 0; i < N; i++) b0[i] = T[i].bytes; uint64_t wb0 = 0; for (int i = 0; i < W; i++) wb0 += T[maxt + i].bytes;
        uint64_t t0 = now_ns(); size_t loads = 0; long f0 = minor_faults();
        if (probe) { while ((double)(now_ns() - t0) < win) { c.cur = chase_loop(c.cur, 4096); loads += 4096; } }
        else usleep((useconds_t)(win / 1000));
        uint64_t t1 = now_ns(); g_faults = minor_faults() - f0;
        uint64_t b1[32]; for (int i = 0; i < N; i++) b1[i] = T[i].bytes;
        __atomic_store_n(&stop, 1, __ATOMIC_RELEASE);
        uint64_t w1 = 0, w0s = 0; for (int i = 0; i < W; i++) w1 += T[maxt + i].bytes;
        for (int i = 0; i < N + W; i++) pthread_join(th[i], NULL);
        sink_ptr(c.cur); (void)w0s;
        double el = (double)(t1 - t0), tot = 0, mn = 1e18, mx = 0;
```

QoS setup (each worker sets the class on itself; the class is read back and stored with every result row):

```
static inline int qos_from_name(const char *s) {
    if (!strcmp(s, "interactive")) return QOS_CLASS_USER_INTERACTIVE;
    if (!strcmp(s, "initiated"))   return QOS_CLASS_USER_INITIATED;
    if (!strcmp(s, "default"))     return QOS_CLASS_DEFAULT;
    if (!strcmp(s, "utility"))     return QOS_CLASS_UTILITY;
    if (!strcmp(s, "background"))  return QOS_CLASS_BACKGROUND;
    return -1;
}
static inline int set_thread_qos(int q) { return q < 0 ? 0 : pthread_set_qos_class_self_np((qos_class_t)q, 0); }
static inline int get_thread_qos(void) { qos_class_t c; int rp; pthread_get_qos_class_np(pthread_self(), &c, &rp); return (int)c; }
```

Dead-code evidence. The unused-result read loop has no symbol in the binary and the timing function calls only the sinking variant:

```
# otool -tV: nm shows read_nosink present? 0 (0 = function removed)
000 000 010 000e7a8 bl 0x10 000eb4c ; symbol stub for: _mach_timebase_info
000 000 010 000e7ac bl 0x10 000eb34 ; symbol stub for: _mach_absolute_time
000 000 010 000e7e0 bl _read_sink
000 000 010 000e7f8 bl 0x10 000eb4c ; symbol stub for: _mach_timebase_info
000 000 010 000e804 bl 0x10 000eb34 ; symbol stub for: _mach_absolute_time
```

Appendix F. AI-use note

What Claude Code generated. The assistant was Claude Code (model claude-sonnet-5-5) running in the terminal on the student's machine, working from the assignment text and instructions supplied by the team. In plain terms it generated: the C benchmark program and its self-tests; the experiment driver, parsing, validation and plotting scripts; the numeric consistency check; the drafts of the analysis text, the hypothesis list and the report assembly; and the choice of substitutes for tools that do not exist on macOS (for example the loaded-

latency harness in place of Intel MLC). It also proposed the experiment designs and the interpretation of each result. It ran the benchmarks on the team's machine.

Errors made by the assistant and corrected during the work. The read kernels first used a single accumulator chain and capped single-thread read bandwidth; the stride sweeps first shrank the number of distinct lines at large strides; the loaded-latency probe first slept during warm-up and lowered its own clock; the first write kernels stored uniform or zero-filled lines, which made stores look faster than random data allows; and several numbers in earlier drafts did not match the data tables. Section 9 describes each in context.

What the students modified. [STUDENT TO FILL: specific edits to code, parameters or text, per member]

What the students tested or verified themselves. [STUDENT TO FILL: runs reproduced, numbers spot-checked, commands run, per member]

What the students rejected or changed after review. [STUDENT TO FILL: suggestions refused, claims removed, results not trusted, per member]

Limits of this note. The assistant cannot know what the students did outside its sessions, so no statement about the students' own work is made here.

Appendix G. Team contribution statement

Names are placeholders. Each member owns the listed experiments and report sections and can explain every number in them from the raw data.

Member	Experiments and review items	Report sections
[Member A], latency and locality	Exp 1, Exp 2 (including stride aliasing), Exp 5 (including plateau tests), core placement, frequency cross-check	2, 3, 6
[Member B], bandwidth and concurrency	Exp 3 (including content test), Exp 4 (loaded latency, knee, Little's Law, read-ceiling tests), peak bandwidth	4, 5
[Member C], misses, TLB and pitfalls	Exp 6, Exp 7 (including layout controls and the huge-page attempt), Exp 8, counters attempt	7, 8, 9
Shared	Hypothesis amendments (each member for their own experiments), rounds and statistics, limitations, proofreading, AI-use note	1, 10 to 13, Appendices
Claude Code	Benchmark and plotting code and fixes, approved re-collection, the numeric consistency check, figure regeneration, report assembly. It proposes verdicts with the evidence; the owning member confirms each verdict.	all (drafting)

Cross-review: [Member A] reviews [Member B]'s experiments, [Member B] reviews [Member C]'s, [Member C] reviews [Member A]'s. Each reviewer reproduces one headline number from the raw data with the provided scripts. [STUDENT TO FILL: reviewer, number reproduced, date, for each pair]

Appendix H. Hardware-counter attempts

Outcome. No counter value was obtained. Hardware counters are inaccessible without administrator rights, and the assistant cannot supply the password. A privileged run is prepared (scripts/run_privileged.sh); the results are reported in Section 8 only if that run was made.

Attempts, in the order specified, with outputs. (1) The PMU event database shipped with macOS lists, for this chip family (as5-1), the events needed: L1D_CACHE_MISS_LD_NONSPEC,

L1D_TLB_MISS_NONSPEC, L2_TLB_MISS_DATA, LD_SRC_LL_CACHE_NONSPEC (loads served by the shared L2) and LD_SRC_MEMSYS_NONSPEC (loads served by the system-level cache or DRAM). (2) Instruments' CPU Counters template, driven through xctrace, runs without administrator rights, but its guided mode exports only cycles, instructions and pipeline-bottleneck fractions with undocumented units; the exported per-core tables contain raw counter arrays without the identity of each counter, and the custom-event configuration is only available through the Instruments window, which a command-line session cannot drive. These tables are not used as evidence anywhere in this report. (3) A counter-reading tool using the private kperf frameworks (src/pmu.h) builds the configuration from the event database successfully but stops at the privileged call:

kpc_force_all_ctrs_set failed (not root?).

```
## 1. PMU event database
$ ls /usr/share/kpep | grep as5
as5-1.plist
as5-2.plist
as5.plist
$ sysctl -n hw.cpufamily  (-> 0xf76c5b1a, maps to as5-1.plist)
−143 893 734
events found in as5-1.plist: L1D_CACHE_MISS_LD_NONSPEC, L1D_TLB_MISS_NONSPEC, L2_TLB_MISS_DATA, LD_SRC_LL_CACHE_NONSPEC,
LD_SRC_MEMSYS_NONSPEC (full list: kpep_
## 2. xctrace / Instruments custom events
$ xcrun xctrace list templates | grep -i counter
CPU Counters
$ xcrun xctrace record --template "CPU Counters" --time-limit 2s --launch -- /usr/bin/true  (default configuration)
Ctrl-C to stop the recording
Target app exited, ending recording…
Recording completed. Saving output file…
Output file saved as: cc.trace
$ xcrun xctrace export --input cc.trace --toc | grep -i schema
schema="CoreTypeByProcess"
schema="CoreTypeByThread"
schema="CounterMetricAggregatedForProcess"
schema="CounterMetricAggregatedForSystem"
schema="CounterMetricByCore"
schema="CounterMetricByThread"
schema="CountingModeSamples"
schema="MetricAggregationForProcess"
schema="MetricAggregationForThread"
schema="MetricTable"
schema="MetricTableForThread"
schema="RemarksByCore"
schema="RemarksByThread"
schema="context-switch"
schema="cpu-narrative"
schema="cpu-state"
schema="dyld-library-load"
schema="gcd-perf-event"
schema="global-poi-layout"
schema="global-roi-layout"
$ strings on the template for configurable event settings
CPU Counters.tracetemplate
## 3. kperf from user space without root (own tool, src/pmu.h)
pmu: kpc_force_all_ctrs_set failed (not root?)
PMU unavailable (needs sudo and kperf)
sudo -n true:
sudo: a password is required
== CounterMetricAggregatedForProcess
<?xml version="1.0"?>
<trace-query-result>
<node xpath='//trace-toc[1]/run[1]/data[1]/table[29]'><schema name="CounterMetricAggregatedForProcess"><col><mnemonic>timestamp</
mnemonic><name>Timestamp</name>
```

Appendix I. Additional tables

Table 13. Kernel cost at the smallest, a middle and the largest footprint (median ns per element, n = 5). The scattered-to-unit ratio depends on the footprint.

kernel	footprint	unit stride ns/elem	stride 64 B ns/elem	scattered ns/elem	scattered / unit	n
z = x*y	12 KiB	0.043	0.244	0.232	5.4x	5
z = x*y	6 MiB	0.097	1.639	1.556	16.1x	5
z = x*y	1.5 GiB	0.123	1.775	10.766	87.9x	5
y = a*x + y	8 KiB	0.045	0.247	0.236	5.2x	5
y = a*x + y	4 MiB	0.077	1.068	1.006	13.1x	5
y = a*x + y	1 GiB	0.079	1.238	6.903	87.4x	5

Table 14. Pointer-chase latency per node versus stride (1 Mi distinct lines). "Implied line rate" is 128 B divided by the latency.

Test	Order	Stride B	Latency	Implied line rate
chase	seq	64	0.9 ns (0.9 to 1.3)	136.4 GB/s line-rate
chase	seq	128	7.8 ns (7.8 to 7.8)	16.4 GB/s line-rate
chase	seq	256	12.0 ns (11.9 to 12.1)	10.7 GB/s line-rate
chase	seq	512	14.3 ns (13.2 to 16.6)	8.9 GB/s line-rate
chase	seq	1024	6.8 ns (6.8 to 7.1)	18.7 GB/s line-rate
chase	seq	2048	103.4 ns (103.0 to 103.8)	1.2 GB/s line-rate
chase	seq	4096	110.6 ns (108.9 to 110.7)	1.2 GB/s line-rate
chase	rand	64	108.0 ns (107.5 to 108.1)	1.2 GB/s line-rate
chase	rand	128	108.4 ns (108.4 to 117.4)	1.2 GB/s line-rate
chase	rand	256	109.1 ns (108.4 to 110.0)	1.2 GB/s line-rate
chase	rand	512	108.3 ns (107.7 to 197.8)	1.2 GB/s line-rate
chase	rand	1024	108.2 ns (107.7 to 108.9)	1.2 GB/s line-rate
chase	rand	2048	109.3 ns (108.5 to 110.4)	1.2 GB/s line-rate
chase	rand	4096	113.6 ns (113.5 to 117.5)	1.1 GB/s line-rate

Table 15. Independent random 16 B loads by stride, with and without a hashed line offset in the stride block.

stride B	power of two	plain GB/s (min to max)	hashed offset GB/s	n rounds
128	yes	2.80 (0.98 to 2.84)		9
192	no	4.69 (2.10 to 4.72)		9
256	yes	2.85 (1.94 to 2.88)	2.84	9
320	no	4.58 (4.08 to 4.85)		9
384	no	2.97 (2.80 to 3.00)		9
512	yes	2.95 (2.35 to 2.97)	3.04	9
640	no	3.12 (2.07 to 3.21)		9
1024	yes	3.20 (2.74 to 3.25)	3.51	9

Table 16. Injection-delay series (median of 12 rounds). The delay series trace the throughput-latency curve below saturation.

series	injected delay (loop iterations)	GB/s	min to max	latency ns	vs idle	N lines	n
6 threads	3 200 000	8	8 to 8	125	1.11x	8	12
6 threads	1 600 000	16	16 to 16	121	1.07x	15	12

series	injected delay (loop iterations)	GB/s	min to max	latency ns	vs idle	N lines	n
6 threads	800 000	31	30 to 31	123	1.09x	30	12
6 threads	400 000	57	56 to 58	126	1.12x	56	12
6 threads	200 000	97	92 to 100	129	1.15x	99	12
6 threads	100 000	162	142 to 166	133	1.18x	168	12
6 threads	60 000	211	173 to 219	131	1.17x	216	12
6 threads	20 000	269	214 to 281	141	1.25x	297	12
6 threads	0	293	216 to 303	152	1.35x	347	12
16 threads	3 200 000	20	15 to 21	138	1.22x	22	12
16 threads	1 600 000	40	35 to 42	137	1.21x	43	12
16 threads	800 000	78	67 to 80	138	1.22x	84	12
16 threads	400 000	140	120 to 146	151	1.34x	165	12
16 threads	200 000	215	138 to 218	181	1.60x	303	12
16 threads	100 000	276	220 to 287	183	1.62x	394	12
16 threads	60 000	284	267 to 295	197	1.74x	436	12
16 threads	20 000	286	269 to 297	206	1.83x	460	12
16 threads	0	285	254 to 296	228	2.03x	509	12

Table 17. Aggregate read GB/s by thread count and variant (median).

variant	1 thr	2 thr	4 thr	6 thr	8 thr	12 thr	16 thr
standard ld1 x4 (e4)	88	163	274	293	266	285	289
ldp q-pairs	94	168	279	292	301	293	293
ld1 four-register	94	170	288	301	301	293	292
background QoS only	33	43	42	43	45	45	46
random buffer content			284	285			292

Table 18. Readers plus two concurrent writer threads.

writers	read threads	read GB/s (min to max)	writer GB/s	total GB/s	n rounds
none	4	274 (236 to 288)	0	274	12
none	6	293 (211 to 302)	0	293	12
2 stnp writers	4	182 (168 to 193)	154	336	7
2 stnp writers	6	205 (189 to 228)	150	355	7
2 st writers	4	176 (165 to 183)	85	261	7
2 st writers	6	218 (196 to 227)	65	282	7

Table 19. A fixed 4 MiB of lines spread over P pages (median ns, n = 12).

pages P holding the same 32 768 lines	original layout	random offsets	constant offset
256	7.6	7.6	7.6
512	8.1	8.1	8.1
1024	10.0	8.5	20.9
2048	22.1	8.8	41.5
4096	33.6	10.3	43.4
8192	40.0	11.9	61.0
16 384	25.1	12.7	102.9

pages P holding the same 32768 lines	original layout	random offsets	constant offset
32768	18.5	13.6	108.1

Table 20. Plateau tests: median ns per load by footprint and node spacing, and the mutating walk.

footprint	64 B spacing	128 B spacing	256 B spacing	mutating walk
48 KiB	0.67	0.67	0.66	2.40
57 KiB	0.67	0.66	0.66	2.41
67 KiB	0.66	0.66	0.66	2.39
80 KiB	0.67	0.66	0.67	2.41
96 KiB	0.67	0.69	0.66	2.40
114 KiB	0.67	0.68	0.67	2.40
135 KiB	1.11	1.12	1.49	2.48
161 KiB	1.96	2.15	2.53	2.79
192 KiB	2.19	2.45	2.62	3.03
228 KiB	2.78	2.58	2.63	3.25
271 KiB	2.88	2.62	2.62	3.45
322 KiB	2.85	2.62	2.64	3.62
384 KiB	2.86	2.68	2.64	3.75
456 KiB	2.87	2.65	2.63	3.92
543 KiB	2.88	2.65	2.65	3.99
645 KiB	2.87	2.66	2.64	4.09
768 KiB	2.91	2.66	2.64	4.18
913 KiB	2.90	2.71	2.67	4.34
1.06055 MiB	5.18	5.14	5.01	4.63
1.26074 MiB	7.19	7.31	7.16	5.31
1.5 MiB	7.23	7.23	7.21	5.90
1.7832 MiB	7.18	7.35	7.25	6.38
2.12109 MiB	7.17	7.18	7.32	6.87
2.52246 MiB	7.16	7.29	7.26	7.22
3 MiB	7.40	7.48	7.40	7.84
3.56738 MiB	7.61	7.64	7.59	8.14
4.24219 MiB	7.75	7.71	7.73	8.55
5.04492 MiB	7.84	7.93	7.92	8.81
6 MiB	7.96	8.03	8.03	9.12

Table 21. Kernel ns per element by element stride at the two largest footprints, plain and hashed offset (n = 7).

kernel	footprint	stride B	power of two	plain ns	hashed ns	n rounds
mul	192 MiB	4	yes	0.09		7
mul	192 MiB	8	yes	0.25		7
mul	192 MiB	16	yes	0.44		7
mul	192 MiB	32	yes	0.89		7
mul	192 MiB	64	yes	1.68		7
mul	192 MiB	96	no	2.19		7
mul	192 MiB	128	yes	15.45		7
mul	192 MiB	192	no	4.25		7

kernel	footprint	stride B	power of two	plain ns	hashed ns	n rounds
mul	192 MiB	256	yes	14.30	16.23	7
mul	192 MiB	320	no	6.56		7
mul	192 MiB	384	no	16.89		7
mul	192 MiB	512	yes	11.46	13.78	7
mul	192 MiB	640	no	16.83		7
mul	192 MiB	1024	yes	8.95	15.44	7
mul	768 MiB	4	yes	0.10		7
mul	768 MiB	8	yes	0.25		7
mul	768 MiB	16	yes	0.44		7
mul	768 MiB	32	yes	0.89		7
mul	768 MiB	64	yes	1.68		7
mul	768 MiB	96	no	2.19		7
mul	768 MiB	128	yes	15.91		7
mul	768 MiB	192	no	4.42		7
mul	768 MiB	256	yes	16.05	17.57	7
mul	768 MiB	320	no	6.47		7
mul	768 MiB	384	no	17.16		7
mul	768 MiB	512	yes	15.90	17.20	7
mul	768 MiB	640	no	17.04		7
mul	768 MiB	1024	yes	16.03	16.85	7
saxpy	128 MiB	4	yes	0.08		7
saxpy	128 MiB	8	yes	0.24		7
saxpy	128 MiB	16	yes	0.31		7
saxpy	128 MiB	32	yes	0.58		7
saxpy	128 MiB	64	yes	1.20		7
saxpy	128 MiB	96	no	1.39		7
saxpy	128 MiB	128	yes	10.98		7
saxpy	128 MiB	192	no	3.41		7
saxpy	128 MiB	256	yes	9.31	10.42	7
saxpy	128 MiB	320	no	4.56		7
saxpy	128 MiB	384	no	10.87		7
saxpy	128 MiB	512	yes	5.95	8.33	7
saxpy	128 MiB	640	no	10.81		7
saxpy	128 MiB	1024	yes	5.16	9.77	7
saxpy	512 MiB	4	yes	0.08		7
saxpy	512 MiB	8	yes	0.24		7
saxpy	512 MiB	16	yes	0.31		7
saxpy	512 MiB	32	yes	0.58		7
saxpy	512 MiB	64	yes	1.21		7
saxpy	512 MiB	96	no	1.43		7
saxpy	512 MiB	128	yes	11.49		7
saxpy	512 MiB	192	no	3.70		7
saxpy	512 MiB	256	yes	11.54	12.19	7
saxpy	512 MiB	320	no	4.94		7
saxpy	512 MiB	384	no	11.53		7
saxpy	512 MiB	512	yes	11.29	11.51	7

kernel	footprint	stride B	power of two	plain ns	hashed ns	n rounds
saxpy	512 MiB	640	no	11.56		7
saxpy	512 MiB	1024	yes	9.84	11.26	7

Table 22. Independent-load useful bandwidth (GB/s), selected points (n = 9).

Order	Access B	Stride B	GB/s
seq	16	64	27.96
seq	16	128	2.87
seq	16	256	2.84
seq	16	1024	2.87
seq	64	64	92.91
seq	64	128	15.76
seq	64	256	11.40
seq	64	1024	11.45
seq	128	128	93.24
seq	128	256	25.47
seq	128	1024	15.78
rand	16	64	4.68
rand	16	128	2.84
rand	16	256	2.89
rand	16	1024	3.27
rand	64	64	6.95
rand	64	128	7.00
rand	64	256	7.11
rand	64	1024	8.16
rand	128	128	7.53
rand	128	256	7.61
rand	128	1024	8.70

Table 23. Loaded latency by mix.

Mix	N threads	app GB/s	probe ns	vs idle	n rounds
100% R	4	273	142	1.26x	6
100% R	16	292	192	1.70x	6
70/30 st	4	246	165	1.48x	6
70/30 st	16	246	549	4.92x	6
50/50 st	4	226	178	1.59x	6
50/50 st	16	225	248	2.22x	6
100% W st	4	167	210	1.84x	6
100% W st	16	165	362	3.18x	6
100% W stnp	4	238	157	1.35x	6
100% W stnp	16	243	286	2.47x	6
50/50 rmw	4	337	193	1.71x	6
50/50 rmw	16	374	251	2.23x	6

Table 24. Fixed 4 MiB of lines over P pages (also Table 19).

P pages	original	random offsets	constant offset
256	7.64	7.62	7.63
512	8.10	8.12	8.12
1024	9.97	8.51	20.88
2048	22.08	8.75	41.54
4096	33.62	10.34	43.38
8192	40.01	11.87	61.05
16 384	25.08	12.74	102.89
32 768	18.51	13.60	108.12